

Azure AI & Copilot

Microsoft Applied Skills

Sie entwickeln praxisnah KI-Anwendungen, Sprach- und Bildanalysen in Azure sowie eigene Copilots – von RAG bis Knowledge Mining. Für Entwickler*innen und Data Scientists, die innovative KI-Lösungen mit Azure AI-Services professionell einsetzen wollen.

Tag 2

Guten
Morgen!

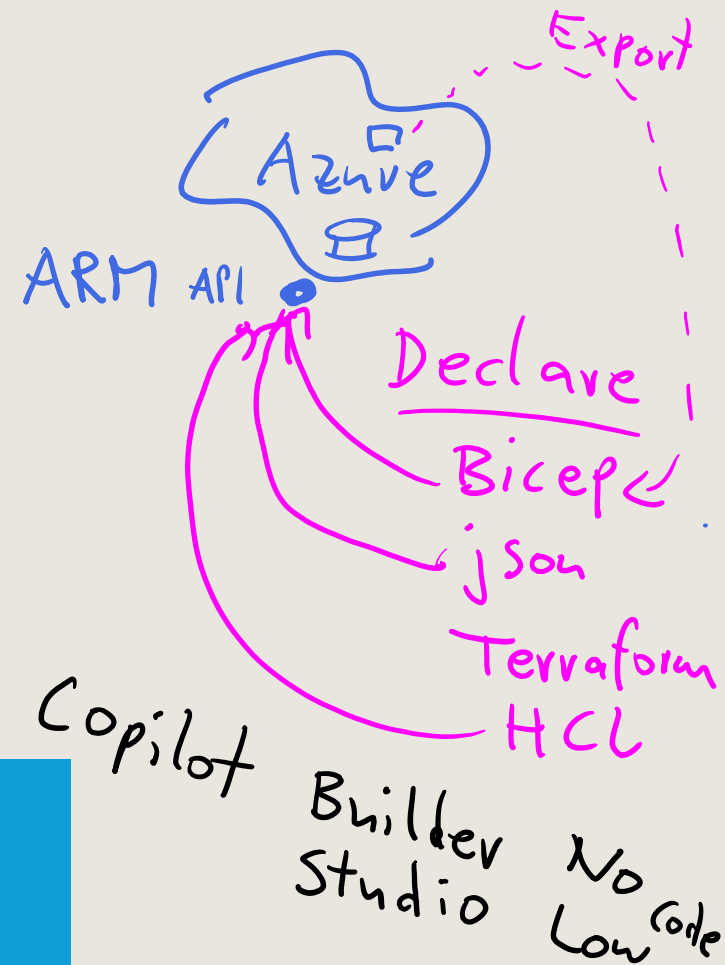
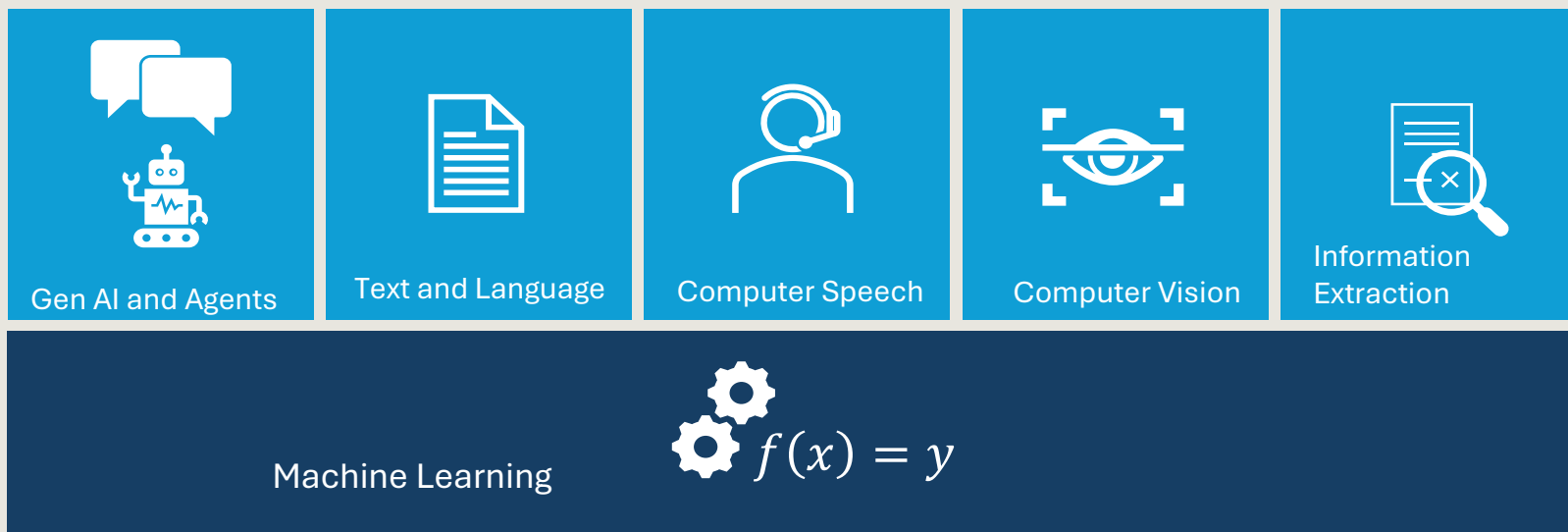
Agenda

MCP
New

Declare
MLP

1. Foundry, Models, Agents
2. VS Code, Python, Chat App
3. Model Context Protocol MCP
4. Language, Speech, Vision, Content Understanding

NLP



A brief history of AI

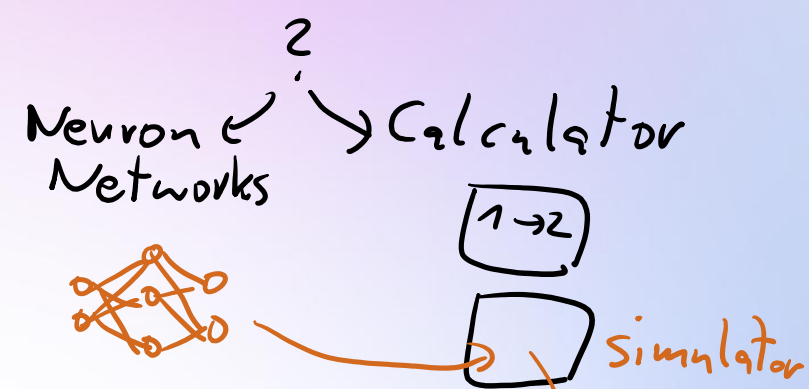
Artificial Intelligence

Machine Learning

Deep Learning

Generative AI

Andrzej Karpatey



1950s

Artificial Intelligence

The field of computer science that seeks to create intelligent machines that can replicate or exceed human intelligence.

1959

Machine Learning

A subset of AI that enables machines to learn from existing data and improve upon that data to make decisions or predictions.

2017

Deep Learning

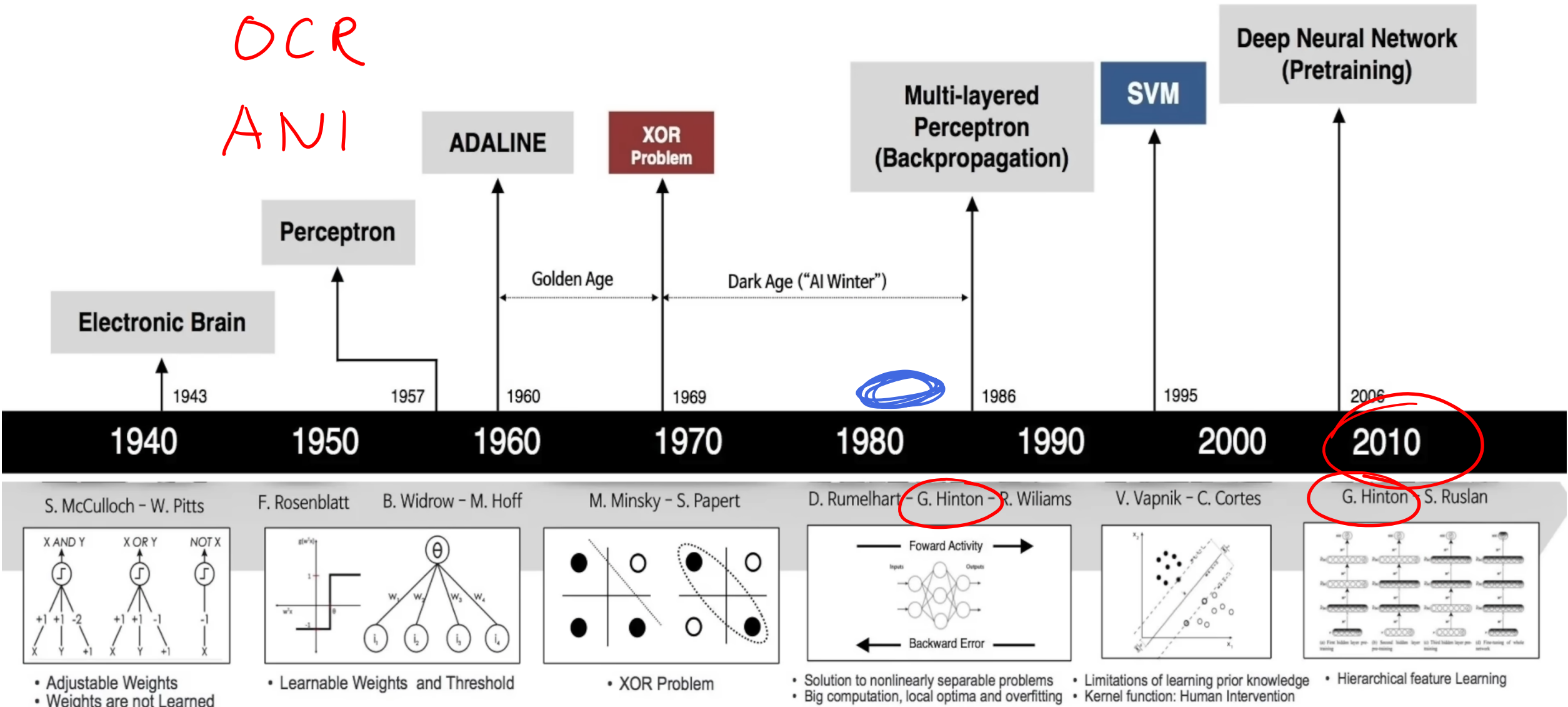
A machine learning technique in which layers of neural networks are used to process data and make decisions.

2021

Generative AI

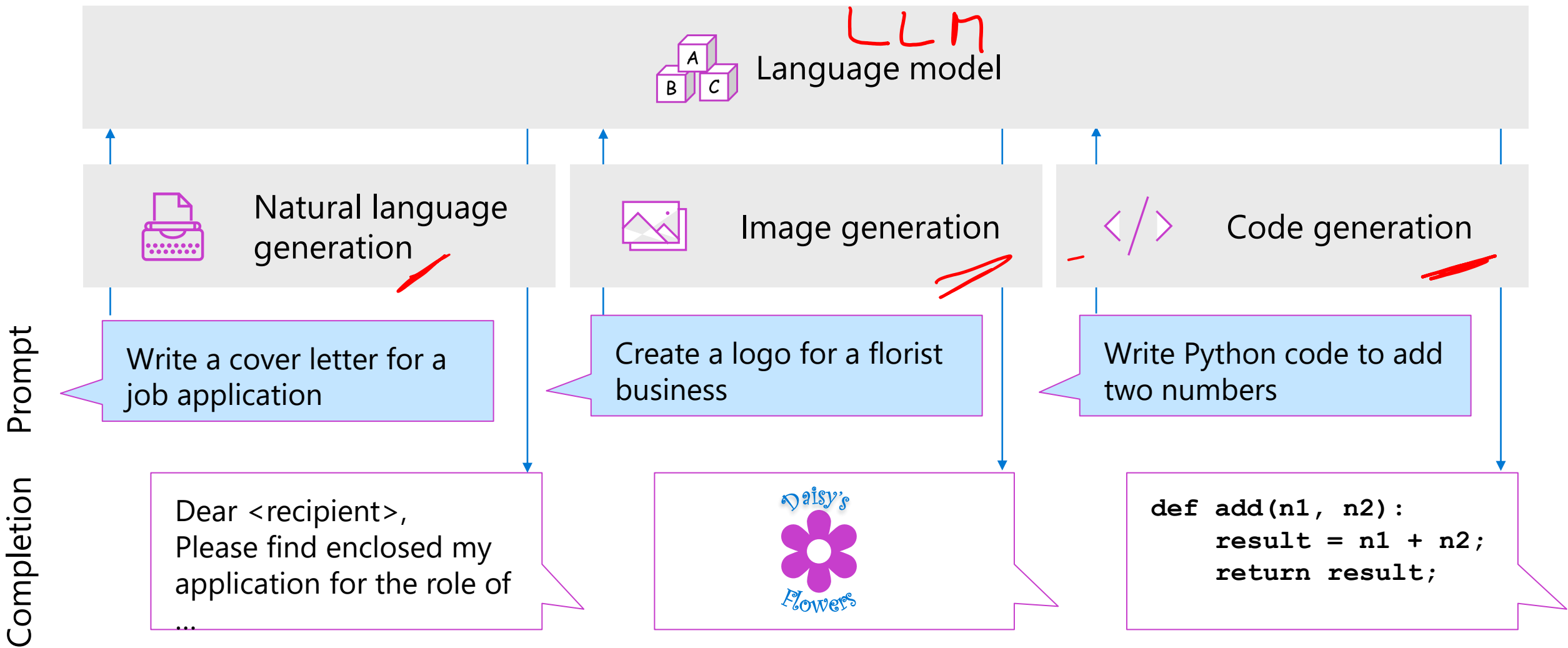
Create new written, visual, and auditory content given prompts or existing data.

OCR
ANI



Brief history of neural network

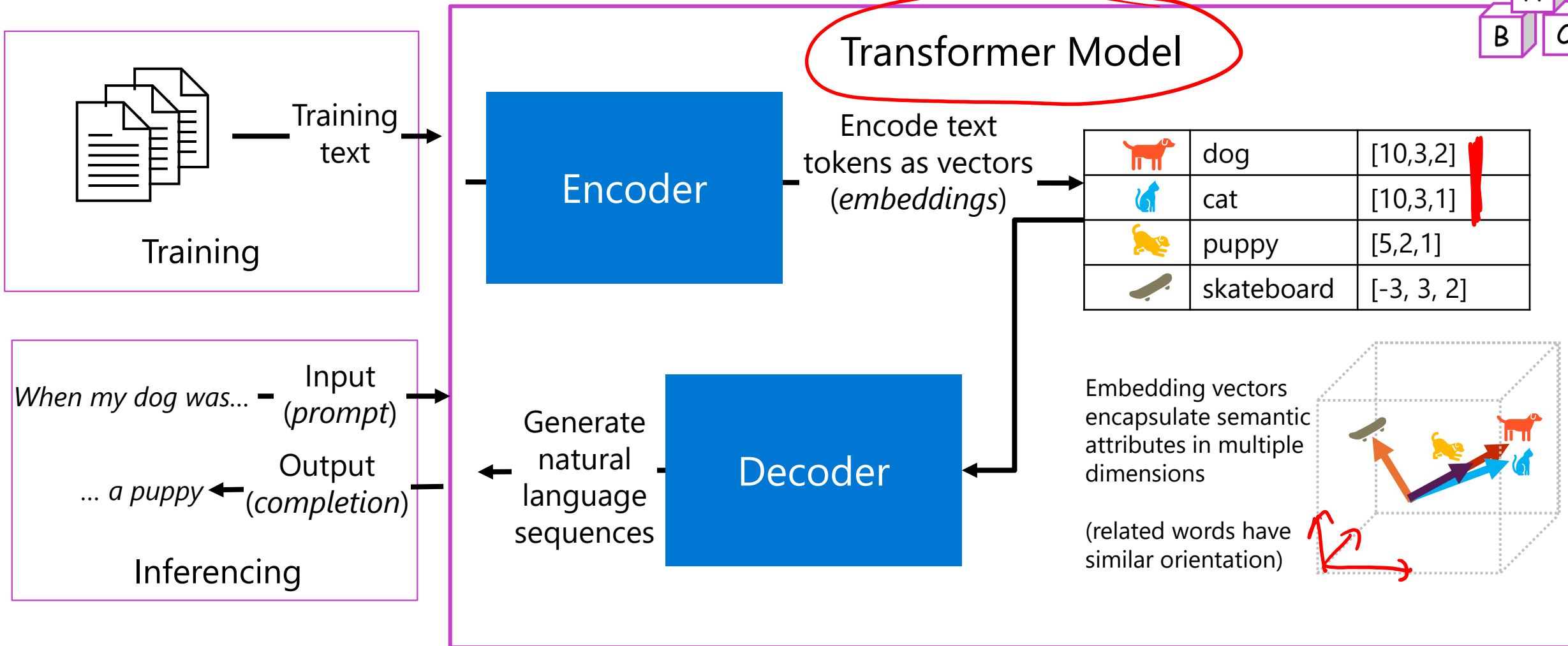
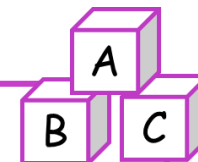
What is Generative AI?



Language models

A (very) high-level overview

RAG



Language models – tokenization

$$12^{30} - 13^{30}$$

Step one: tokenization

The first step in training a transformer model is to decompose the training text into *tokens*.

Example sentence: *I heard a dog bark loudly at a cat.*

"I"=1

"heard"=2

"a"=3

"dog"=4

"bark"=5

"loudly"=6

"at"=7

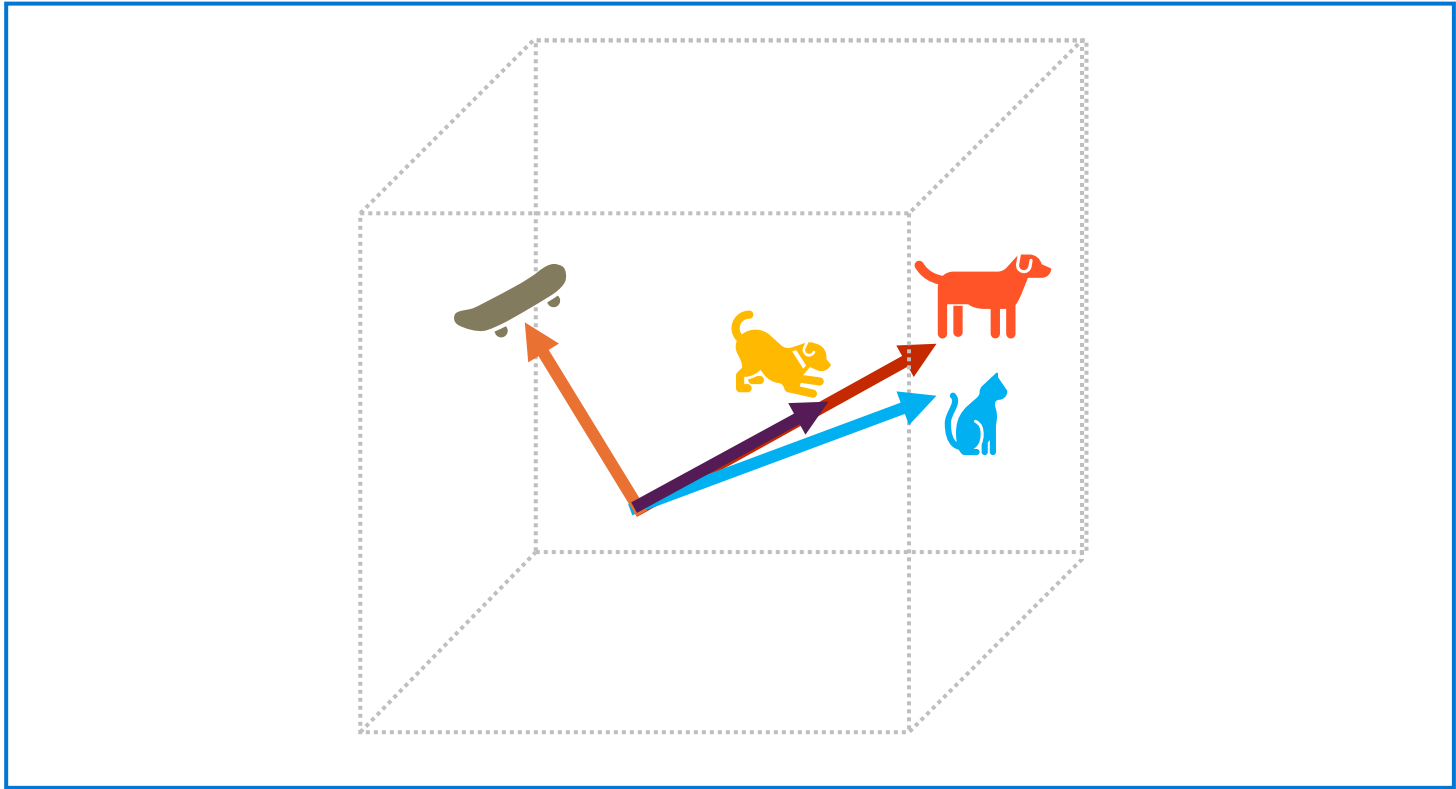
"cat"=8

- The sentence is now represented with the tokens: $[1\ 2\ 3\ 4\ 5\ 6\ 7\ 3\ 8]$.
- Note "a" is tokenized as 3 only once.
- Similarly, the sentence "I heard a cat" could be represented with the tokens $[1\ 2\ 3\ 8]$.

Language models – embeddings

Step two: embeddings

The semantic relationship between tokens is encoded in vectors, known as embeddings.



	Token	Word	Embedding
	4	dog	[10,3,2]
	8	cat	[10,3,1]
	9	puppy	[5,2,1]
	10	skateboard	[-3, 3, 2]

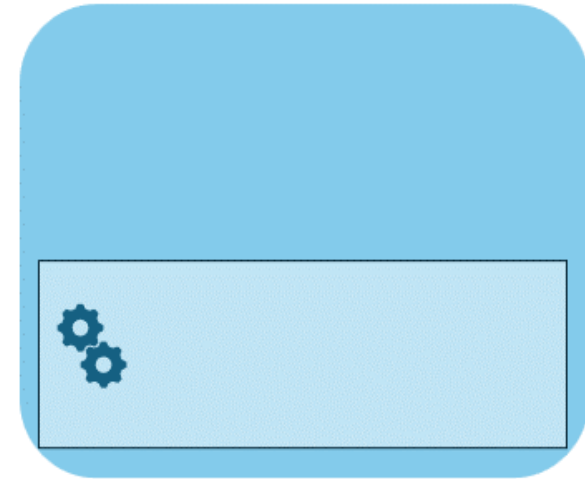
Language models – attention

Step three: attention

Capture the strength of relationships between tokens using the attention technique.

Example:

- Goal: predict token after "**dog**."
- Represent "**I heard a dog**" as vectors.
- Assign "**heard**" and "**dog**" more weight.
- Several possible tokens can come after dog.
- The most probable token is added to the sequence, in this case "**bark**."



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to

Attention Visualizations

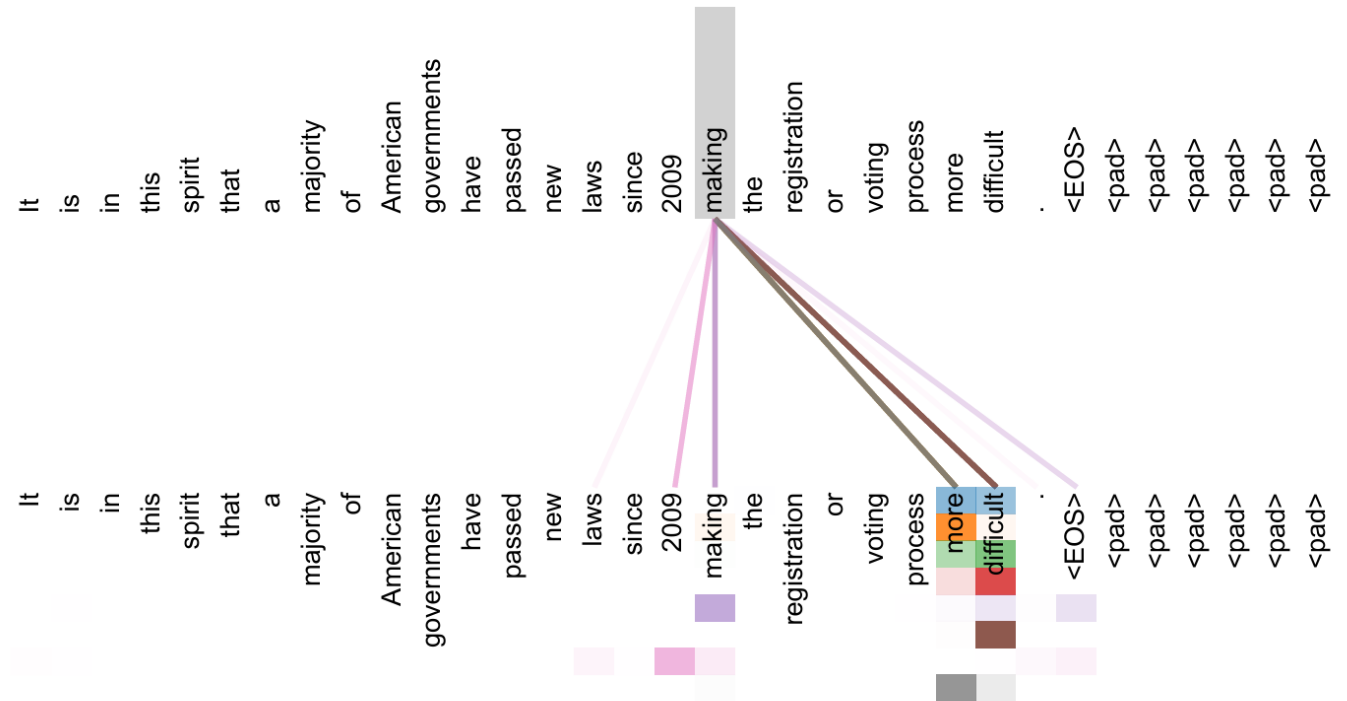
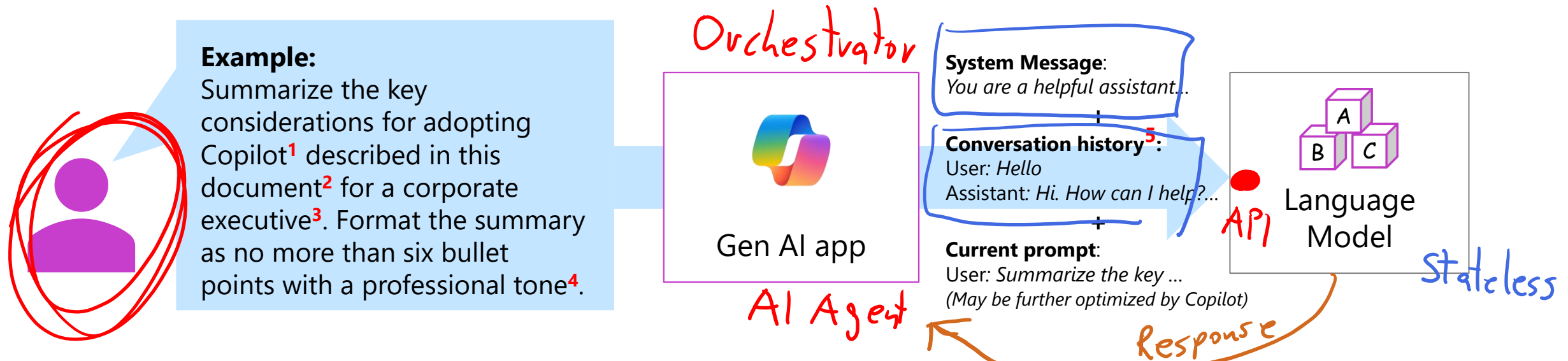


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.

Considerations for prompts



1. Start with a specific goal for what you want the output to be
2. Provide a source to *ground* the response in a specific scope of information
3. Add context to maximize response appropriateness and relevance
4. Set clear expectations for the response
5. Iterate based on previous prompts and responses to refine the result

Language models

Large and small language models

Large language models (LLMs)	Small language models (SLMs)
Trained with large volumes of general text data	Trained with focused text data
Many billions of parameters	Fewer parameters
Comprehensive language generation capabilities in multiple contexts	Focused language generation capabilities in specialized contexts
Large size can impact performance and portability	Fast and portable
Time-consuming (and expensive) to fine-tune with your own training data	Faster (and less expensive) to fine-tune with your own training data
Examples include: Azure OpenAI in Microsoft Foundry GPT 5 Mistral 7B Meta Llama 3	Examples include: Microsoft Phi 4 Microsoft Orca 2 Hugging Face GPT Neo

...

MAI

What are agents?

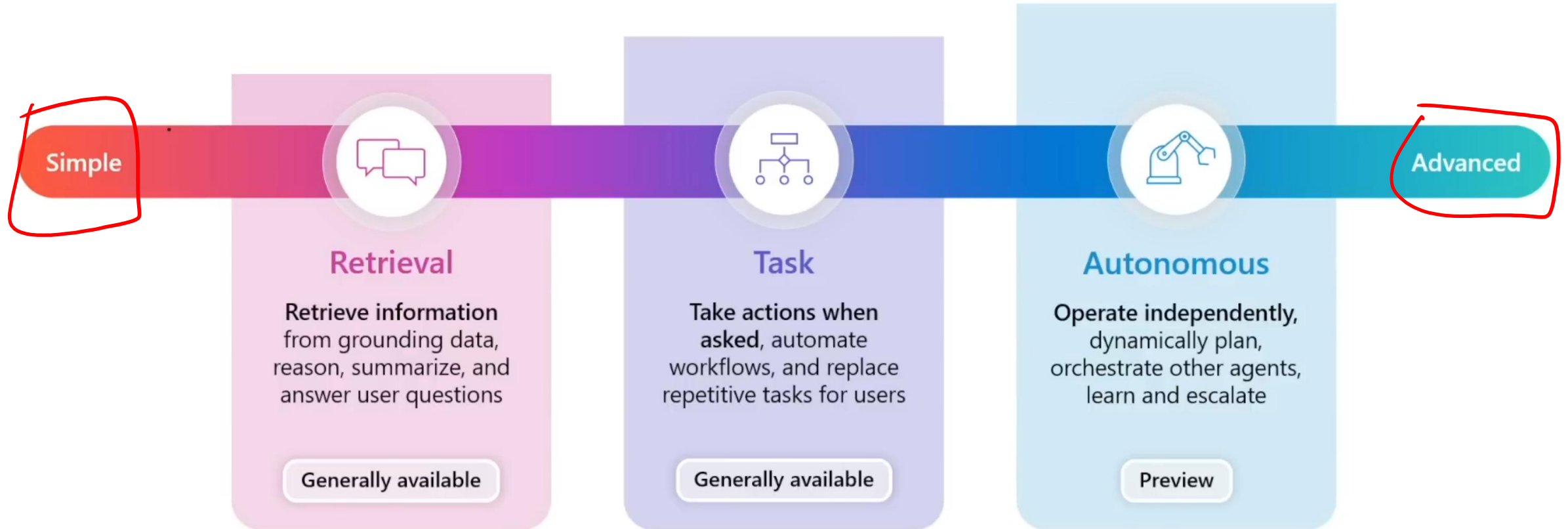
Agents are generative AI applications that can respond to user input or assess situations *autonomously* and take appropriate *actions*.

Agents have three core components:

- **Model (LLM):** Powers reasoning and language understanding
- **Instructions:** Define the agent's goals, behavior, and constraints
- **Tools:** Let the agent retrieve *knowledge* or take *action*

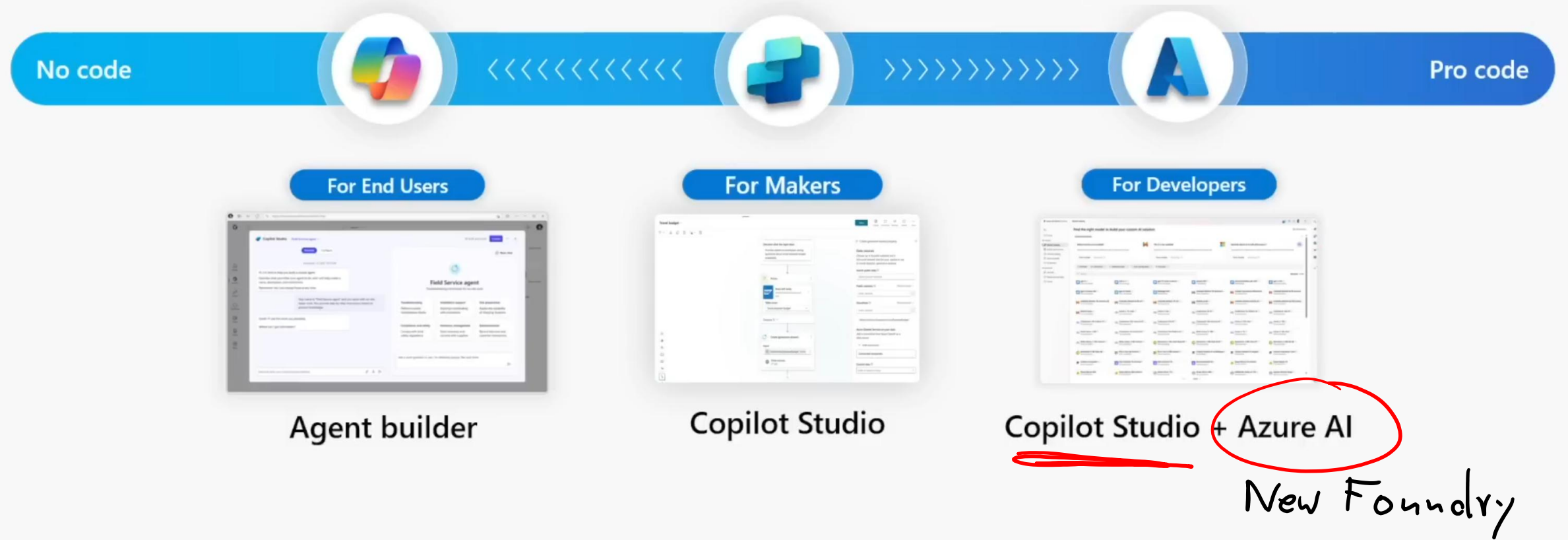


Spectrum of agents



← Agents vary in levels of complexity and capabilities depending on your need →

Agent tooling





Microsoft Applied Skills

Learn more at: learn.microsoft.com/credentials

Don't just know it. Show it. Demonstrate your ability to use Microsoft technologies in real-world AI and cloud scenarios. Choose from technical and new **business-focused credentials**.

— Technical
— Business



Beginner

Cloud & AI Platforms

AI Business Solutions

Security

Create an AI agent

Get started with classes, properties, and methods in C#

Get started with cloud security and monitoring tasks

Get started with Azure management tasks

Generate reports by using AI research agents

Streamline business workflows with AI chat

Get started with identities and access using Microsoft Entra

New

Get started developing agents in Microsoft Foundry

Intermediate

Accelerate app development by using GitHub Copilot

Configure secure access to your workloads using Azure networking

Implement a Real-Time Intelligence solution with Microsoft Fabric

New

Resolve GitHub issues by using GitHub Copilot

Automate Azure Load Testing by using GitHub Actions

Create and manage canvas apps with Power Apps

Defend against cyberthreats with Microsoft Defender XDR

Secure AI solutions in the cloud

Administer Active Directory Domain Services

Deploy and configure Azure Monitor

Implement a data warehouse in Microsoft Fabric

New

Integrate model context protocol tools with agents in Microsoft Foundry

Create agents in Microsoft Copilot Studio

Create and manage model-driven apps with Power Apps and Dataverse

Implement information protection and data loss prevention by using Microsoft Purview

Secure Azure services and workloads with Microsoft Defender for Cloud regulatory compliance controls

Build a natural language processing solution with Azure AI Language

Deploy cloud-native apps using Azure Container Apps

Implement security through a pipeline using Azure DevOps

Create and manage automated processes by using Power Automate

~~Prepare security and compliance support Copilot for Microsoft 365~~

Implement retention, eDiscovery, and Communication Compliance in Microsoft Purview

Coming soon
Protect information in Microsoft 365 Copilot by using Microsoft Purview

Build AI-powered solutions by using Microsoft Azure Database for PostgreSQL

Develop data-driven applications by using Microsoft Azure SQL Database

Migrate SQL Server workloads to Azure SQL Database

Build a generative AI chat app

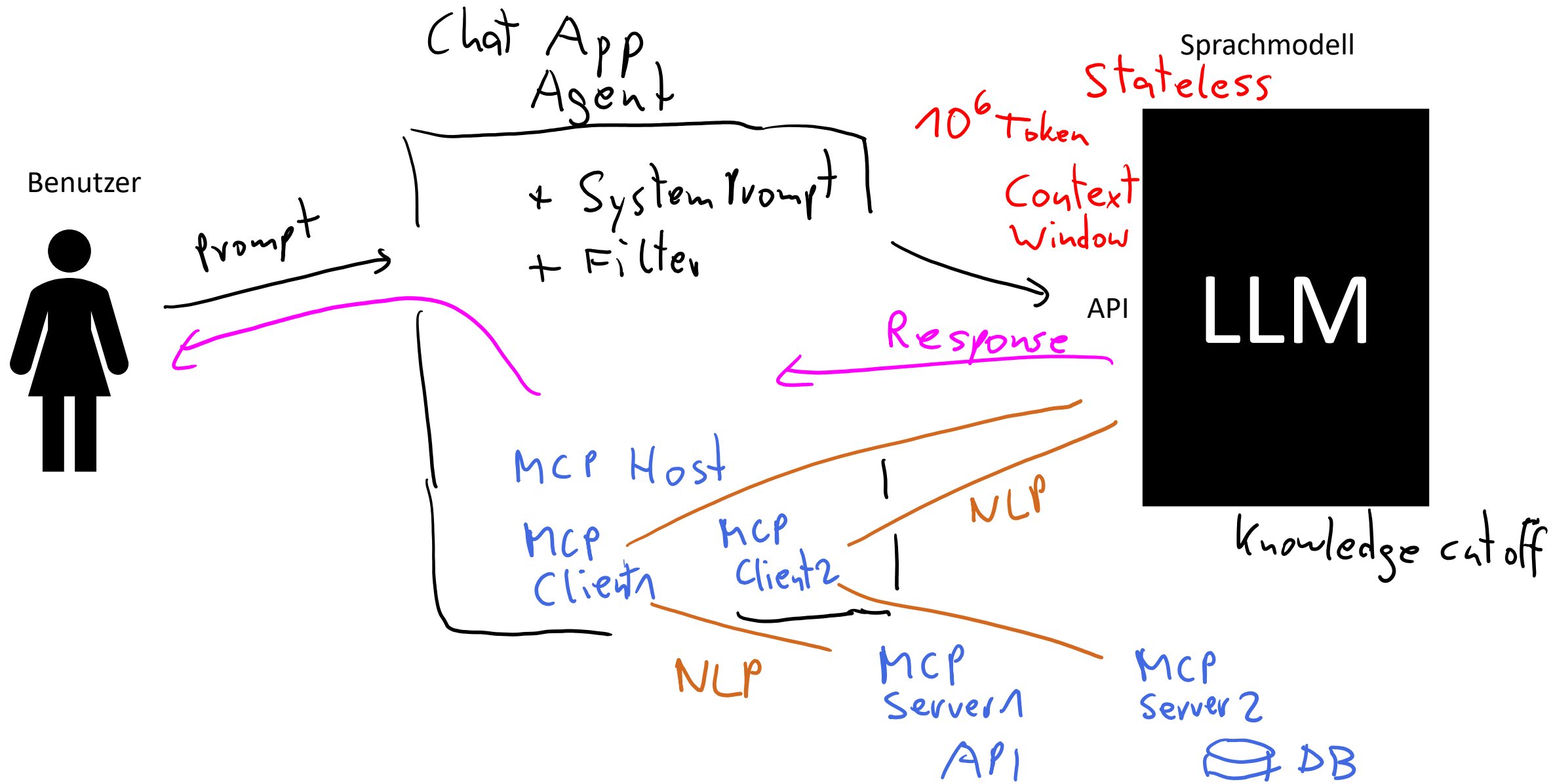
Enhance agents with autonomous capabilities

Secure storage for Azure Files and Azure Blob Storage

Configure and migrate to Azure Database for PostgreSQL

will retire
06/30/2026

Architektur





Developing Azure AI Apps and Agents

<https://aka.ms/mslearn-develop-ai-agents>

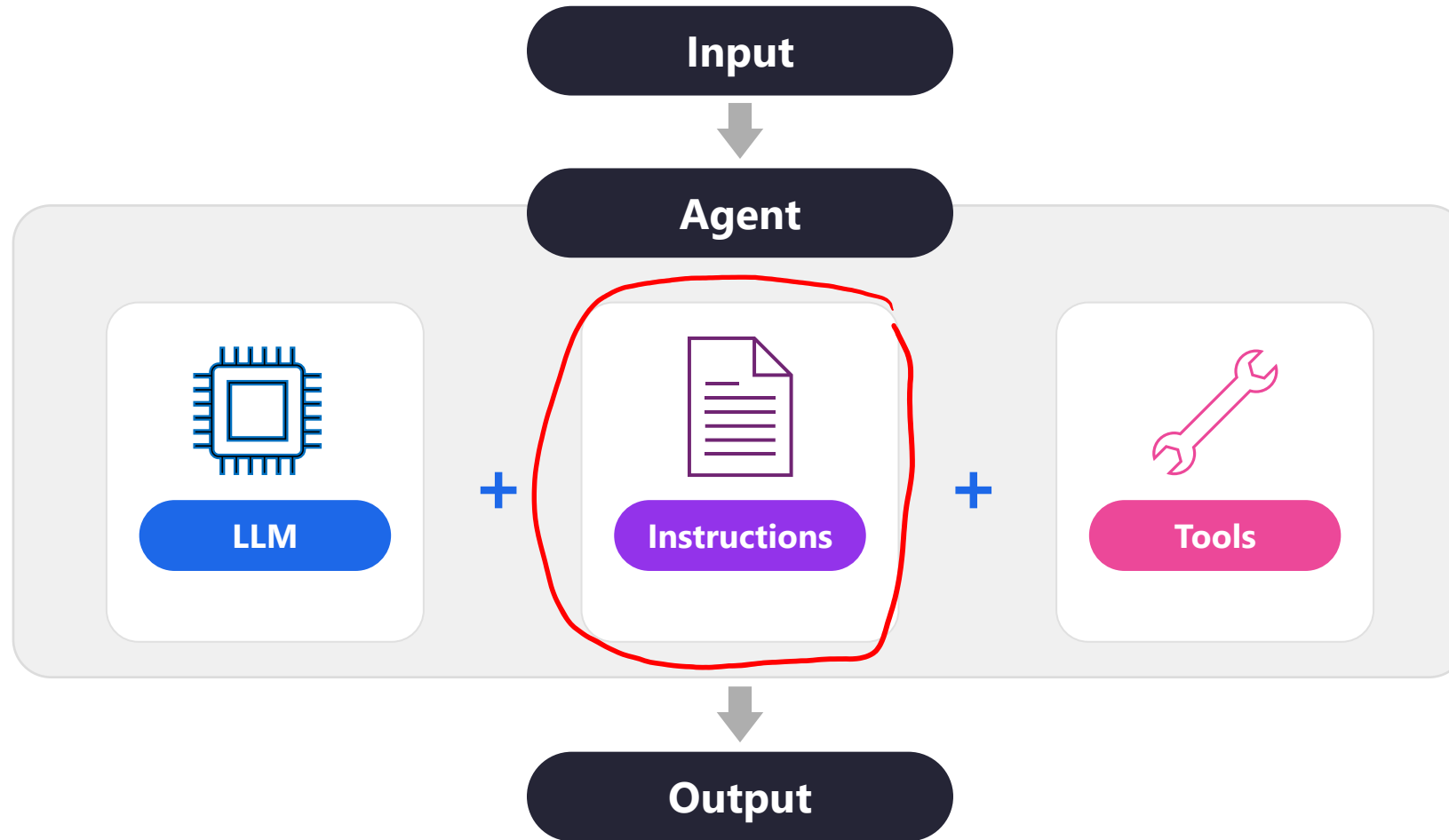


Develop AI agents with Microsoft Foundry and Visual Studio Code

<https://aka.ms/develop-agents-vs-code>

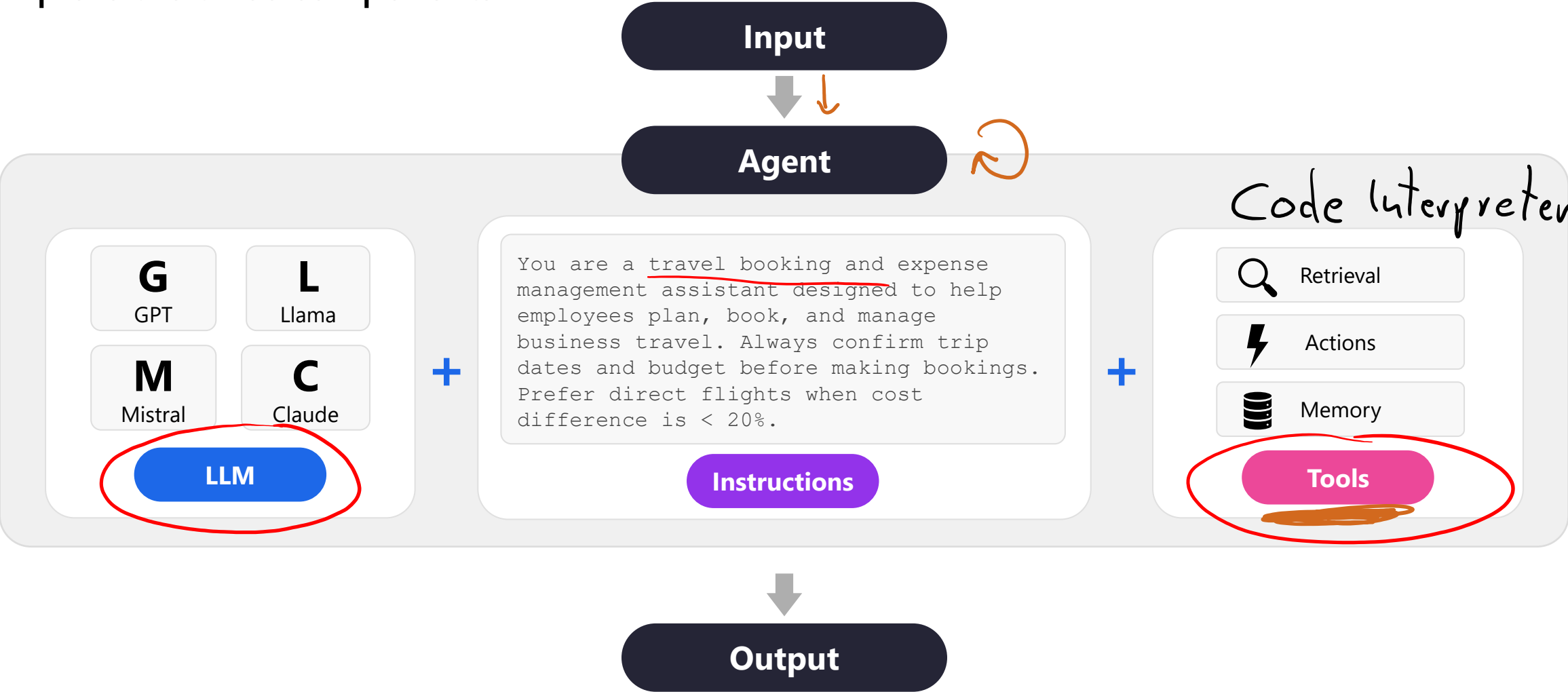


Understand AI agents



Understand AI agents

Explore the three components



Why agents are useful

Alignment ?



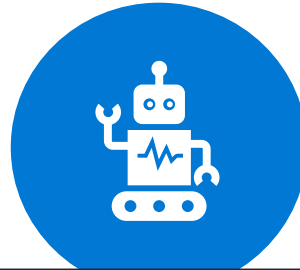
Automation of routine tasks

Handles repetitive activities, freeing humans to focus on strategic and creative work.



Scalability

Grows capabilities without proportional increases in headcount or operational costs.



Agent



Enhanced decision-making

Analyzes data to surface insights, predict outcomes, and recommend actions in real time.



24/7 availability

Operates continuously without breaks, keeping services available around the clock.

Open Claw
Peter Steinberger

Claude

AI agent use cases

Personal productivity

Schedule our team standup for next week.

Done! Mon–Fri at 9 AM — invites sent to all 6 team members.

Also draft a recap email from yesterday's notes.

Draft ready in your Drafts folder. Want me to send it?

Sales

Find leads in the Pacific Northwest retail sector.

Found 14 leads. Drafted personalized emails and flagged 3 as high-priority for calls.

Schedule the priority calls for this week.

Calls booked Tuesday and Thursday — calendar invites sent.

Research

Any major shifts in the AI market this week?

3 key updates: AI adoption up 40%, EU AI Act enforcement started, healthcare AI funding at record high.

Generate a summary report for the team.

Report generated — 2-page summary with source links attached.

Customer service

I need a refund for order #4821.

Refund approved! \$47.99 will return to your card in 2–3 business days.

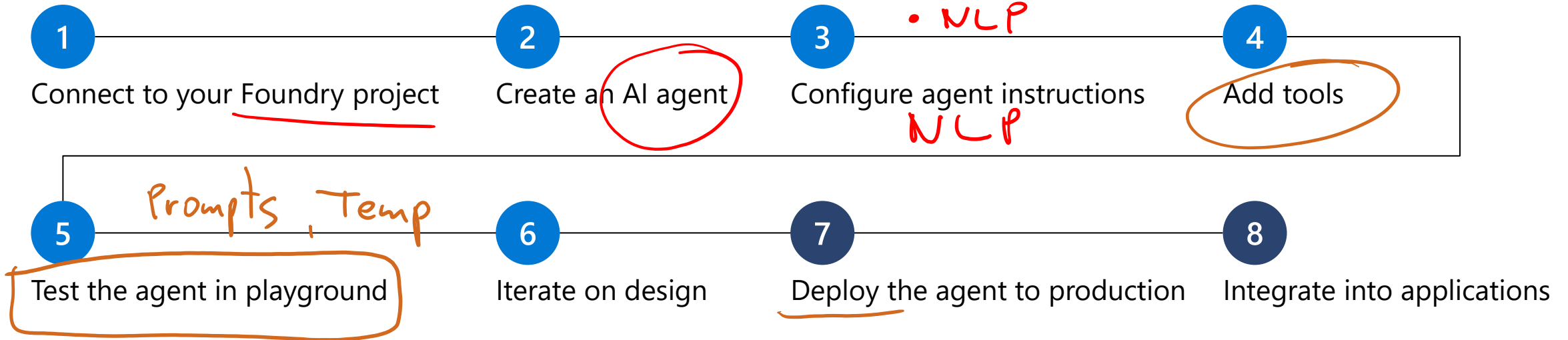
Can I get a confirmation email?

Sent to your registered email. Is there anything else I can help with?

Typical development workflow

4 User Interface

- Lockkarte
- Terminal *az vm list -o table*
- GUI Portals
- NLP



✓ Required resources

- **Microsoft Foundry project**
Organizes agents, models, and assets
- **Model deployment**
For example, GPT-4.1 or Claude Sonnet 4.6

+ Optional resources

- **Azure AI Search** Knowledge retrieval for Foundry IQ or File Search
- **Azure Storage** Files agents can read and write
- **Azure Key Vault** Securely store secrets and credentials
- **Azure Functions** Custom tool implementations and business logic

Agent

Choose your development approach



Foundry portal

Visual, web-based, no local setup required

- ✓ **Quick prototyping**
Test agent concepts without setting up a local development environment
- ✓ **Visual configuration**
Configure agents through intuitive forms and dropdowns rather than code
- ✓ **Centralized management**
View and manage all agents across projects in one place with dashboards
- ✓ **Team collaboration**
Share configurations with stakeholders who prefer visual interfaces



Visual Studio Code

Developer-centric, code-integrated, Git-friendly

- ✓ **Developer-centric workflows**
Build agents alongside application code in a single familiar environment
- ✓ **Version control integration**
Track agent configurations in Git alongside your codebase
- ✓ **Code-first development**
Edit YAML configurations directly and integrate agents into application logic
- ✓ **Local development**
Design and test agents offline before deploying to Azure

OR

Python

VS Code
Erich Gamma
Editor
Terminal
Extensions

Many teams use both: the Foundry portal for initial exploration and reviews, and VS Code for detailed development and production deployments.

Exercise

Build AI agents with portal and VS Code

In this exercise, you'll:

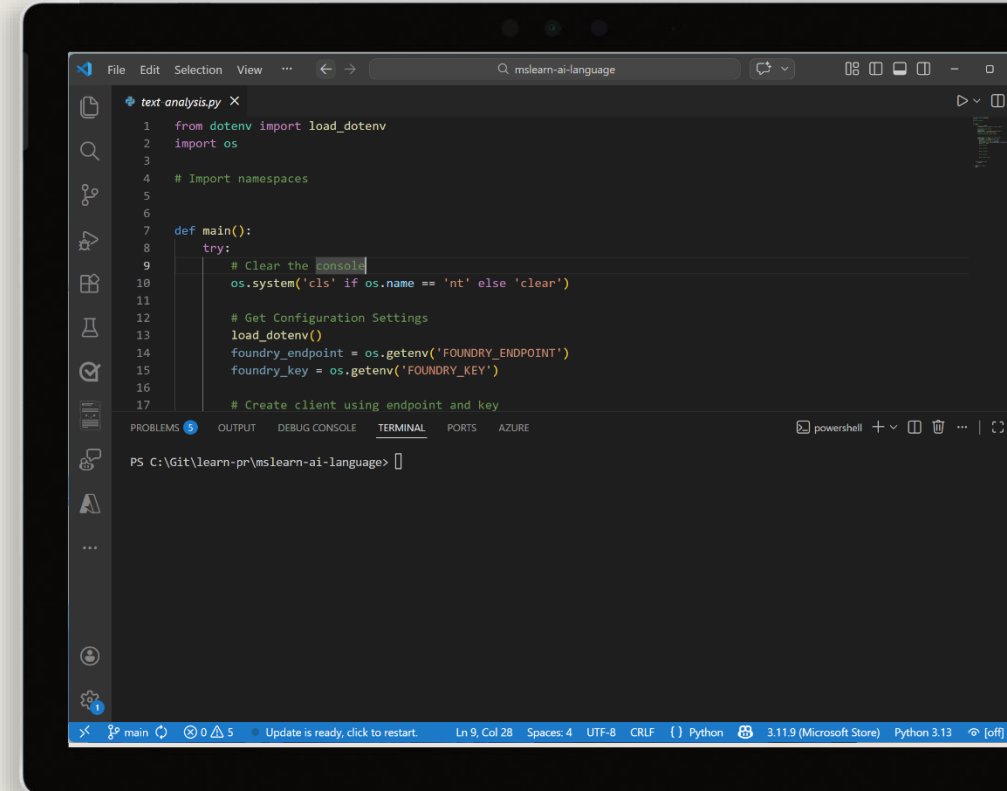
- Create an AI agent in Microsoft Foundry portal
- Interact with your agent using VS Code

Start the exercise at:

6. Build AI agents with portal and VS Code

🕒 1 h

Go Deploy



Knowledge check



1 What is the primary benefit of using Microsoft Foundry Agent Service compared to building agents with standard APIs?

- ☐ It provides access to more powerful AI models
- ☐ It requires no Azure subscription
- ☒ It handles tool calling, state management, and infrastructure automatically
- ☐ It only works with the Azure portal

2 How does Microsoft Foundry Agent Service handle conversation state?

- ☐ By requiring developers to manually manage conversation history
- ☐ Through external database connections
- ☒ Through the Responses API which automatically manages conversation context
- ☐ Using local file storage on the client device

Lab 3

Email → Applied Skill

Develop a generative AI chat
app with Microsoft Foundry

<https://aka.ms/mslearn-develop-chat>



Choosing an endpoint and SDK

`.venv` | Scripts | `activate.ps1`

Endpoint	Foundry Project <code>https://{resource-name}.services.ai.azure.com/api/projects/{project-name}</code>	Azure OpenAI <code>https://{resource-name}.openai.azure.com/openai/v1</code>
SDK	Microsoft Foundry SDK <code>pip install azure-ai-projects openai</code>	OpenAI SDK <code>pip install openai</code>
Authentication	Microsoft Entra ID <code>pip install azure-identity</code>	Microsoft Entra ID or API key <code>pip install azure-identity</code>
Chat API	Responses API Foundry direct models only	Responses or ChatCompletions API All Foundry models
Best for	Foundry-specific capabilities with OpenAI-compatible interfaces for Foundry direct models	Latest OpenAI SDK models and features with the full OpenAI API surface
<pre>from azure.identity import DefaultAzureCredential from azure.ai.projects import AIProjectClient from openai import OpenAI project_client = AIProjectClient(credential=DefaultAzureCredential(), endpoint=FOUNDRY_PROJECT_ENDPOINT) openai_client = project_client.get_openai_client(api_version="2024-10-21")</pre>		<pre>from openai import OpenAI openai_client = OpenAI(api_key=API_KEY_OR_TOKEN, base_url=AZURE_OPENAI_ENDPOINT)</pre>

Compare ChatCompletions with Responses API

	 ChatCompletions API	 Responses API
State management	Client-side - developer stores and sends the full messages array	Server-side - API stores the thread automatically
System prompt	First entry in messages array: <code>{"role": "system", "content": "..."} <u>role</u> → <u>content</u></code>	Dedicated parameter: <code>instructions=..."</code>
Payload per request	Grows with conversation length  →	Constant - only latest input + an ID 
Accessing the response	<code>completion.choices[0].message.content</code>	<code>response.output_text</code>

Compare per-turn code pattern



ChatCompletions API



1

INITIAL
STATE

```
conversation_messages = [
    {"role": "system", "content": "You are helpful..."}]
```

2

ADD
INPUT

```
conversation_messages.append({
    "role": "user",
    "content": input_text})
```

3

CALL
API

```
completion = openai_client.chat.completions.create(
    model="gpt-4.1",
    messages=conversation_messages)
```

4

GET
OUTPUT

```
assistant_msg = completion.choices[0].message.content
print(assistant_msg)
```

5

UPDATE
STATE

```
conversation_messages.append({
    "role": "assistant",
    "content": assistant_msg})
```



Responses API

```
last_response_id = None
```

```
response = openai_client.responses.create(
    model="gpt-4.1",
    instructions="You are helpful...",
    input=input_text,
    previous_response_id=last_response_id)
```

```
print(response.output_text)
```

```
last_response_id = response.id
```

Using the *Responses* API

```
# Track previous response
last_response_id = None

# Loop until the user wants to quit
print("Assistant: Enter a prompt (or type 'quit' to exit)")
while True:
    input_text = input('\nYou: ')
    if input_text.lower() == "quit":
        print("Assistant: Goodbye!")
        break

    # Get a response to the user's prompt
    response = openai_client.responses.create(
        model="gpt-4.1",
        instructions="You are a helpful AI assistant that explains technology concepts.",
        input=input_text,
        previous_response_id=last_response_id
    )
    assistant_text = response.output_text
    print("\nAssistant:", assistant_text)
    # Track the response for the next turn
    last_response_id = response.id
```

```
Assistant: Enter a prompt (or type 'quit' to exit)
You: What is machine learning?
Assistant: Machine learning is a type of artificial intelligence (AI) that enables
computers to learn from data and predict unknown values based on learned patterns.
You: Can you give me an example?
Assistant: Certainly! A simple example of supervised learning would be a model that
predicts house prices based on features like size, location, and number of rooms.
You: quit
Assistant: Goodbye!
```

Using the *ChatCompletions* API

```
# Initial system message
conversation_messages=[
    {
        "role": "system", "content": "You are a helpful AI assistant that answers questions and provides information."
    }
]
# Loop until the user wants to quit
print("Assistant: Enter a prompt (or type 'quit' to exit)")
while True:
    input_text = input('\nYou: ')
    if input_text.lower() == "quit":
        print("Assistant: Goodbye!")
        break
    # Add user message
    conversation_messages.append(
        {"role": "user", "content": input_text}
    )
    # Get completion
    completion = openai_client.chat.completions.create(
        model="gpt-4.1",
        messages=conversation_messages
    )
    assistant_message = completion.choices[0].message.content
    print("\nAssistant:", assistant_message)
    # Add assistant message
    conversation_messages.append(
        {"role": "assistant", "content": assistant_message}
    )
```

```
Assistant: Enter a prompt (or type 'quit' to exit)
You: When was Microsoft founded?
Assistant: Microsoft was founded on April 4, 1975.
You: Who founded it?
Assistant: Microsoft was founded by Bill Gates and Paul Allen.
You: quit
Assistant: Goodbye!
```

Exercise

Create a generative AI chat app

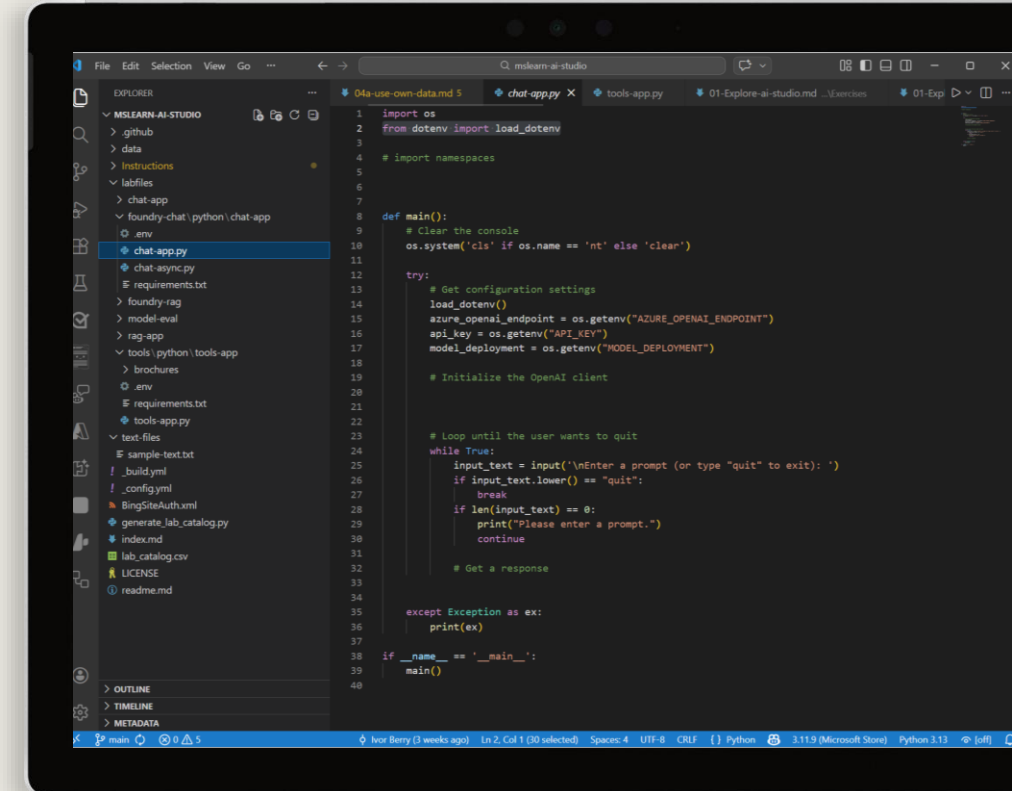
In this exercise, you'll:

- Create a Microsoft Foundry project
- Deploy a model
- Get the endpoint and key
- Create a client application to chat with the model

Start the exercise at:

3. Create a generative AI chat app

🕒 1 h



Azure AI & Copilot

Microsoft Applied Skills

Sie entwickeln praxisnah KI-Anwendungen, Sprach- und Bildanalysen in Azure sowie eigene Copilots – von RAG bis Knowledge Mining. Für Entwickler*innen und Data Scientists, die innovative KI-Lösungen mit Azure AI-Services professionell einsetzen wollen.

Tag 3

Guten
Morgen!

Agenda

HuggingFace

MAI

New

Phi

AI Apps

Declare

SDK — App¹.py

1. Foundry, Models, Agents

2. VS Code, Python, Chat App

3. Model Context Protocol MCP

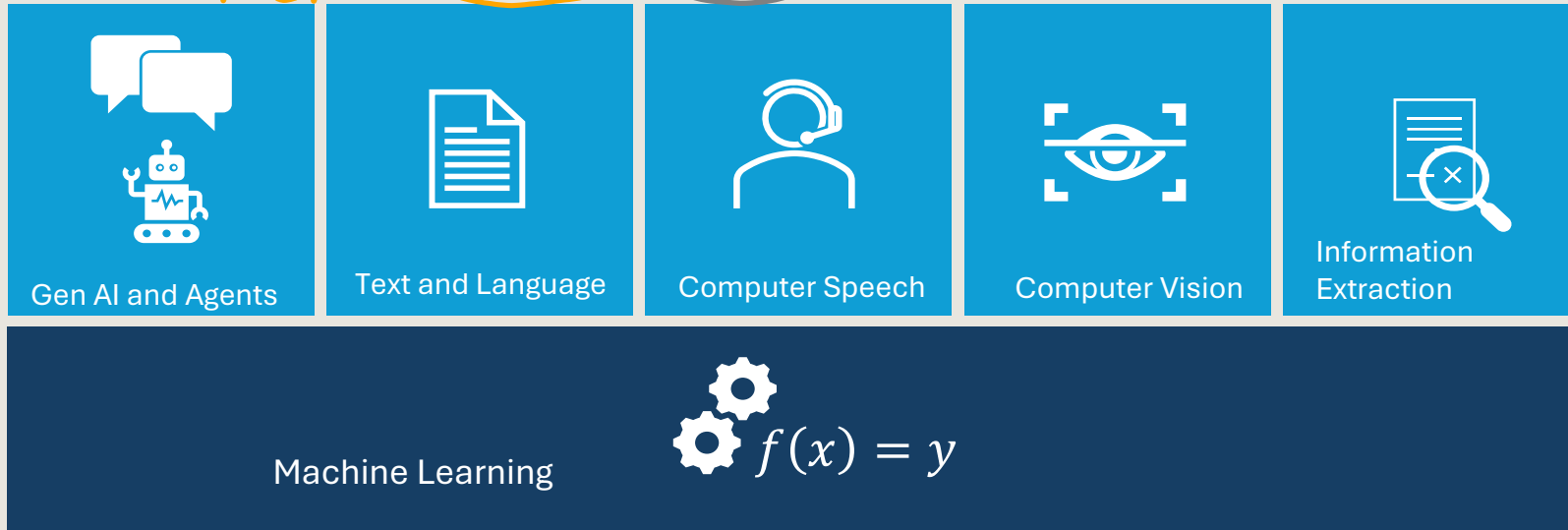
4. Language, Speech, Vision, Content Understanding OCR

Applied Skills

App¹.py ←

App².py ←

NLP





APPLIED SKILLS

Microsoft Applied Skills: Build a generative AI chat app

Validate your technical skills and open doors to new possibilities of advancement with Microsoft Applied Skills.

What is the Microsoft Foundry SDK?



Tip

You can use the OpenAI chat client provided by a Microsoft Foundry project to chat with any model deployed in the associated Microsoft Foundry resource - even non-OpenAI models, such as Microsoft Phi models.

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from openai import AzureOpenAI

try:

    # connect to the project
    project_endpoint = "https://....."
    project_client = AIProjectClient(
        credential=DefaultAzureCredential(),
        endpoint=project_endpoint,
    )

    # Get a chat client
    chat_client = project_client.get_openai_client(api_version="2024-10-21")

    # Get a chat completion based on a user-provided prompt
    user_prompt = input("Enter a question:")

    response = chat_client.chat.completions.create(
        model=your_model_deployment_name,
        messages=[
            {"role": "system", "content": "You are a helpful AI assistant."},
            {"role": "user", "content": user_prompt}
        ]
    )
    print(response.choices[0].message.content)

except Exception as ex:
    print(ex)
```

1.) Appl. Skill

App1.py
App2.py

.env
.env



2.)

Go Deploy

Lab 3

Project

- API Endpoint

- API Key

- Model phi-4
(gpt-4-mini)

VS Code

requirement.txt ←

pip install -r requirements.txt

python App1.py

(.venv)

(.venv)

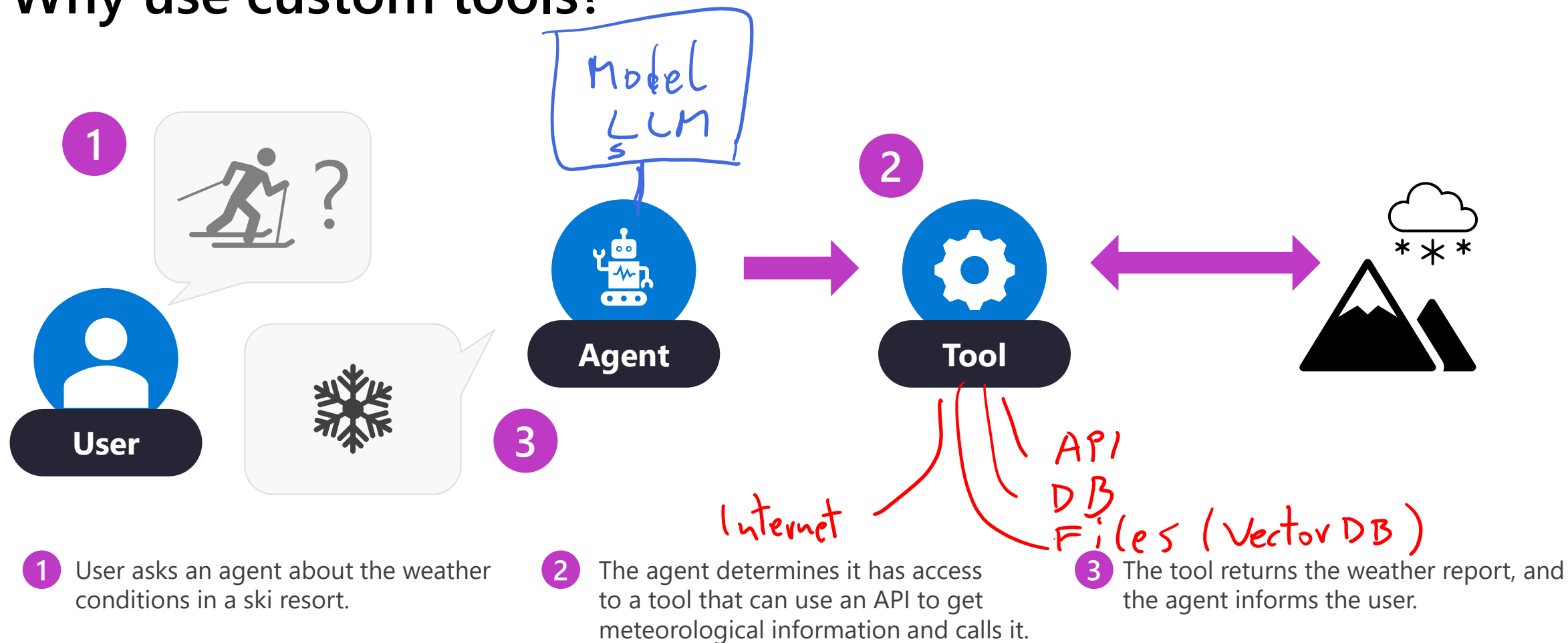


Integrate custom tools into your agent

<https://aka.ms/mslearn-agent-custom-tools>



Why use custom tools?



Options for implementing custom tools



Custom function

Describe custom functions to an agent and let it dynamically identify and call the right function with the correct arguments.

Supports multiple programming languages for integrating custom logic and workflows.

Function calling



Azure Functions

Build intelligent, event-driven applications with minimal overhead.

Triggers determine when a function executes; bindings streamline connections to input and output data sources.

Event-driven

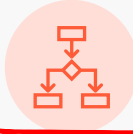


OpenAPI specification

Build intelligent, event-driven applications with minimal overhead.

Triggers determine when a function executes; bindings streamline connections to input and output data sources.

OpenAPI 3.0

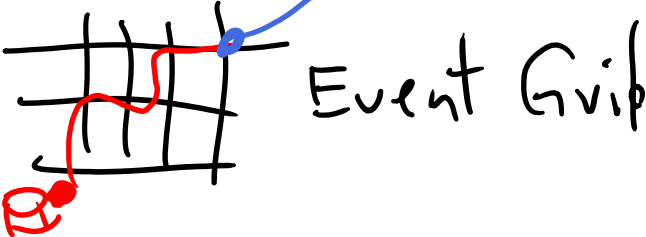


Azure Logic Apps

Build intelligent, event-driven applications with minimal overhead.

Triggers determine when a function executes; bindings streamline connections to input and output data sources.

Low-code



Exercise

Build an agent with custom tools

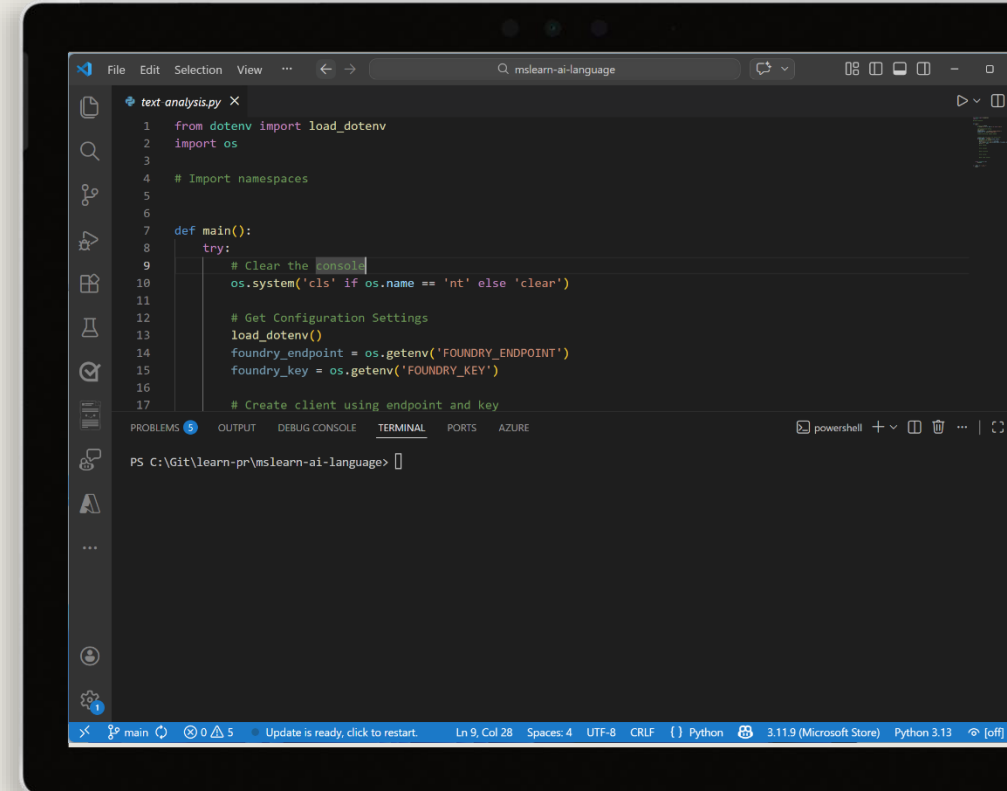
In this exercise, you'll:

- Create a function for the agent to use
- Define the function tools
- Create the agent that uses the function tools
- Send a message to the agent and process the response
- Process function calls and display the agent's response

Start the exercise at:

7. Use a custom function in an AI agent

🕒 1 h



Knowledge check



- 1** What are custom tools, and how can they help you develop effective agents?
 - ☒ Callable functions that an agent can use to extend its capabilities
 - ☐ Extensions for Visual Studio Code to create and deploy agents
 - ☐ Fine-tuned models that the agent uses to generate custom output

- 2** You need to integrate functionality from an OpenAPI 3.0-based web service. What should you do?
 - ☐ Add the JSON schema to the agent's instructions
 - ☐ Rewrite the web service as a Python function and hard-code it
 - ☒ Add the web service as an OpenAPI specification tool to the agent definition

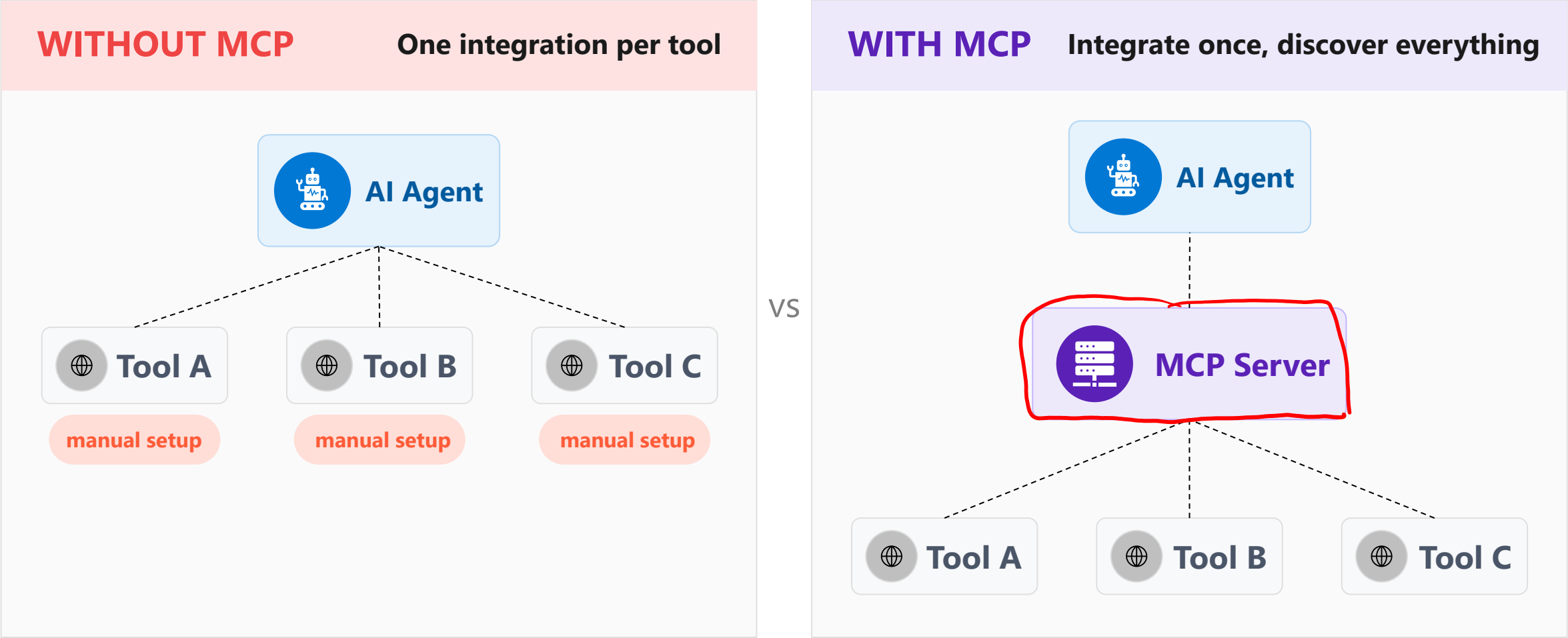
USB
↓

Integrate MCP tools with Azure AI agents

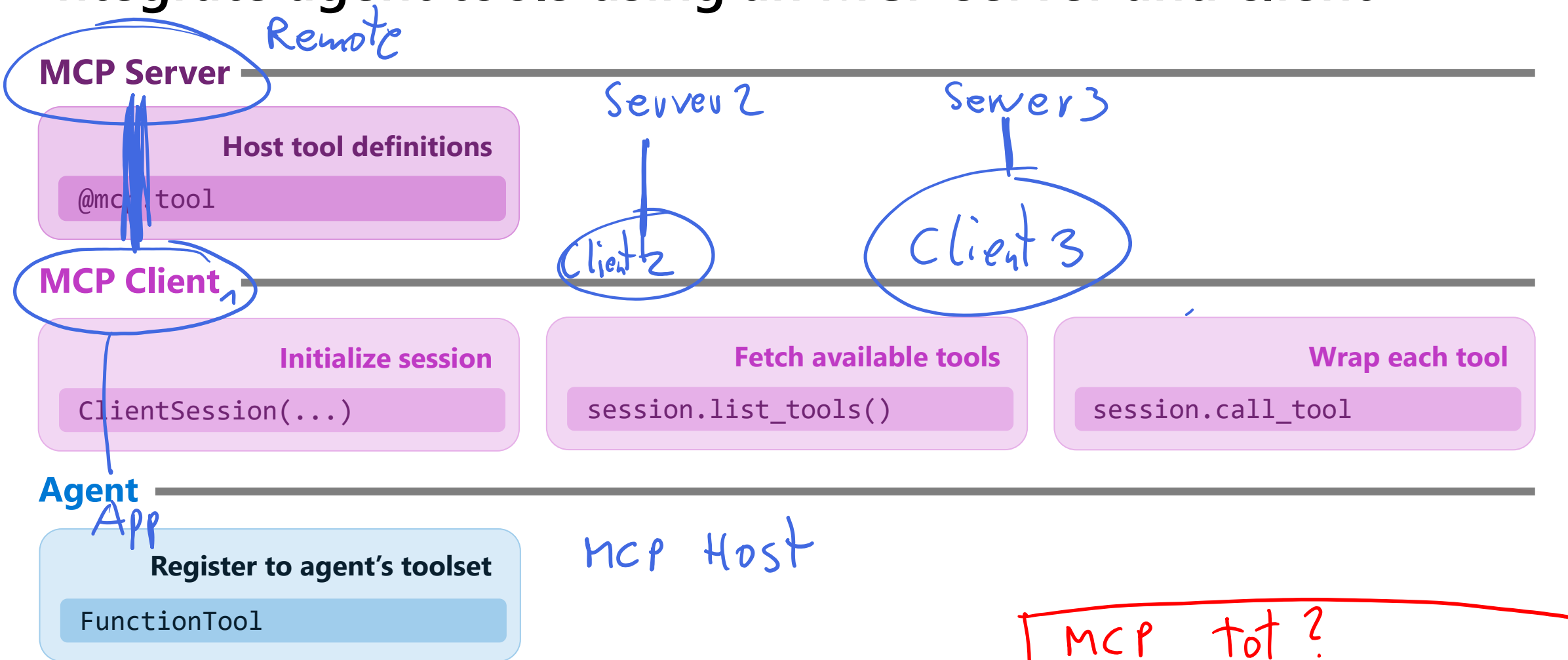
<https://aka.ms/mslearn-agent-mcp>



Understand Model Context Protocol for AI agents



Integrate agent tools using an MCP server and client



Exercise

Connect MCP tools to Foundry Agents

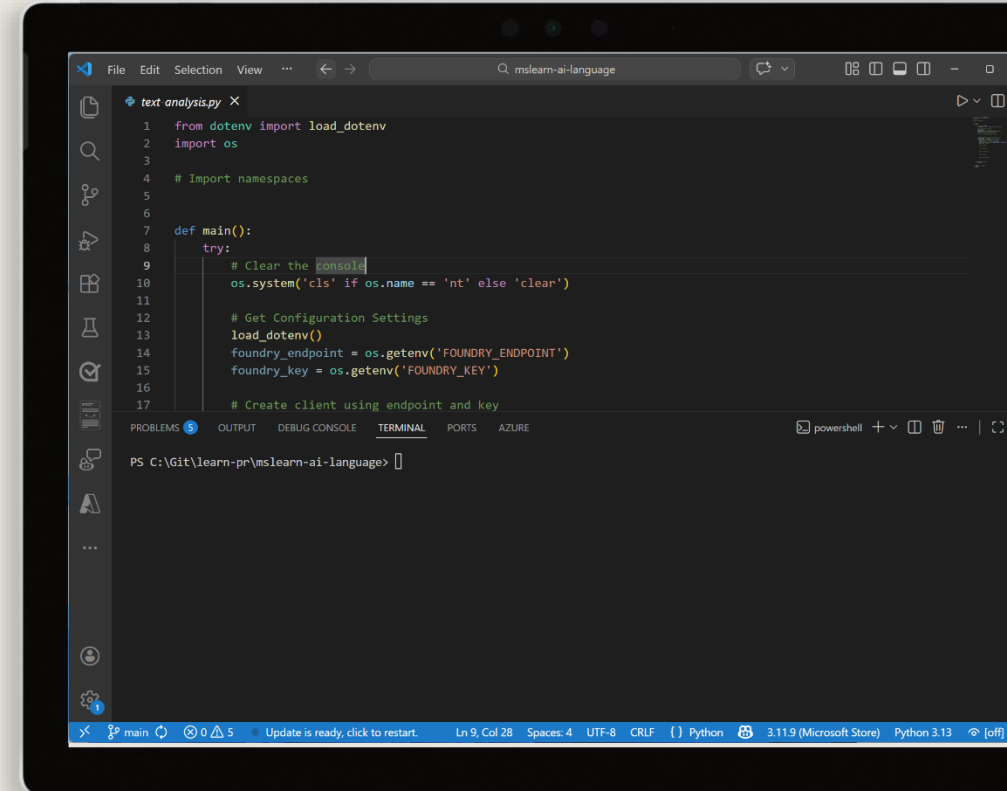
In this exercise, you'll:

- Connect a Foundry Agent to a remote MCP server
- Connect a Foundry Agent to custom MCP server tools

Start the exercise at:

8. Develop an AI agent with Model Context Protocol (MCP) tools

🕒 1 h



Knowledge check



- 1** What role does the MCP server play in the MCP agent tool integration?
- ☐ Runs the AI agent and processes user prompts directly
 - ☐ Manages network connections between multiple agents
 - ☒ Hosts tool definitions and makes them available for discovery by the client

- 2** How does an MCP client retrieve available tools from the MCP server?
- ☒ By calling `session.list_tools()` to get the current tool catalog
 - ☐ By reading a static JSON file from the server directory
 - ☐ By subscribing to server events via a WebSocket connection

Build knowledge-enhanced AI agents with Foundry IQ

<https://aka.ms/mslearn-agent-foundry-iq>

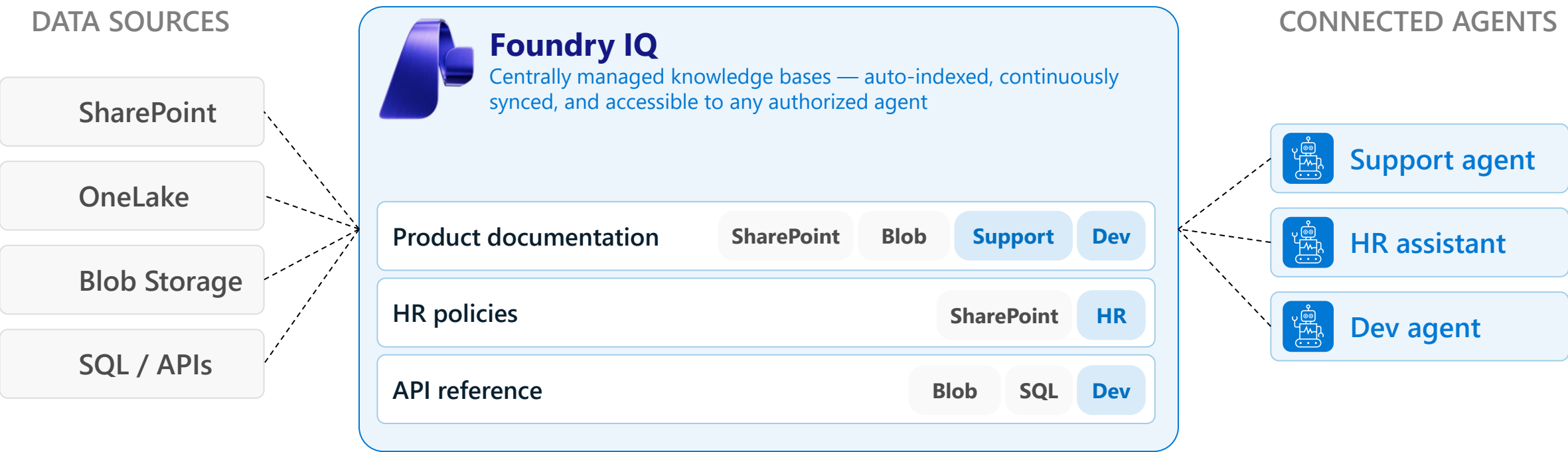


Retrieval Augmented Generation (RAG) for agents

LIMITATION	Simple agent	RAG-powered agent
Knowledge cutoff	Outdated answers Unaware of anything past its training cutoff date	Always current Retrieves live documents from connected knowledge bases
No private data	Generic only No access to internal procedures or product specs	Org-specific Searches internal knowledge bases for company content
Lack of context	Irrelevant advice Ignores org workflows, approval chains, and security policies	Contextually aware Grounds advice in retrieved org policies and workflows
Fabricated responses	Compliance risk Confident-sounding answers with no source to verify	Verifiable sources Cites the exact document behind every response
Scalability	Duplicated effort Every team rebuilds the same RAG pipeline independently	Shared platform One knowledge platform (e.g., Foundry IQ) serves all teams

Explore Foundry IQ

A managed knowledge platform built into Microsoft Foundry
Define your knowledge bases once, connect any number of agents to them



Configure data sources



Indexed

Pre-processedHigh relevanceCustom

Azure AI Search Index

BEST FOR
Enterprise search with custom ingestion pipelines and AI enrichment

SharePoint (Indexed)

BEST FOR
Advanced search on SharePoint content with custom query pipelines



Direct

No index neededRaw file access

Azure Blob Storage

BEST FOR
Document files in Azure Storage — PDFs, DOCX, text, and more

OneLake

BEST FOR
Unstructured data and documents in Microsoft Fabric's unified data lake



Real-time

Always currentLive retrievalNo data copy

Web

BEST FOR
Current public information retrieved via Bing at query time

SharePoint (Remote)

BEST FOR
Live SharePoint content with Microsoft 365 governance and permissions

Configure retrieval with Foundry IQ

Effective instructions specify three critical behaviors

● When to retrieve

Tell the agent to always use the knowledge base, never rely on training data

● How to cite

Specify the exact format for source attribution

● What to do when unsure

Define fallback behavior when information isn't found

```
retrieval_instructions = """You are a helpful HR assistant.
```

CRITICAL RULES:

- - You must ALWAYS search the knowledge base before answering any question
- You must NEVER answer from your own knowledge or training data
- - Every answer MUST include citations
- - If the knowledge base doesn't contain the answer, respond with "I don't have that information in our current documentation. Please contact HR directly at hr@company.com"

```
Your role is to provide accurate, verifiable information from company documentation."""
```

Exercise

Integrate an AI agent with Foundry IQ

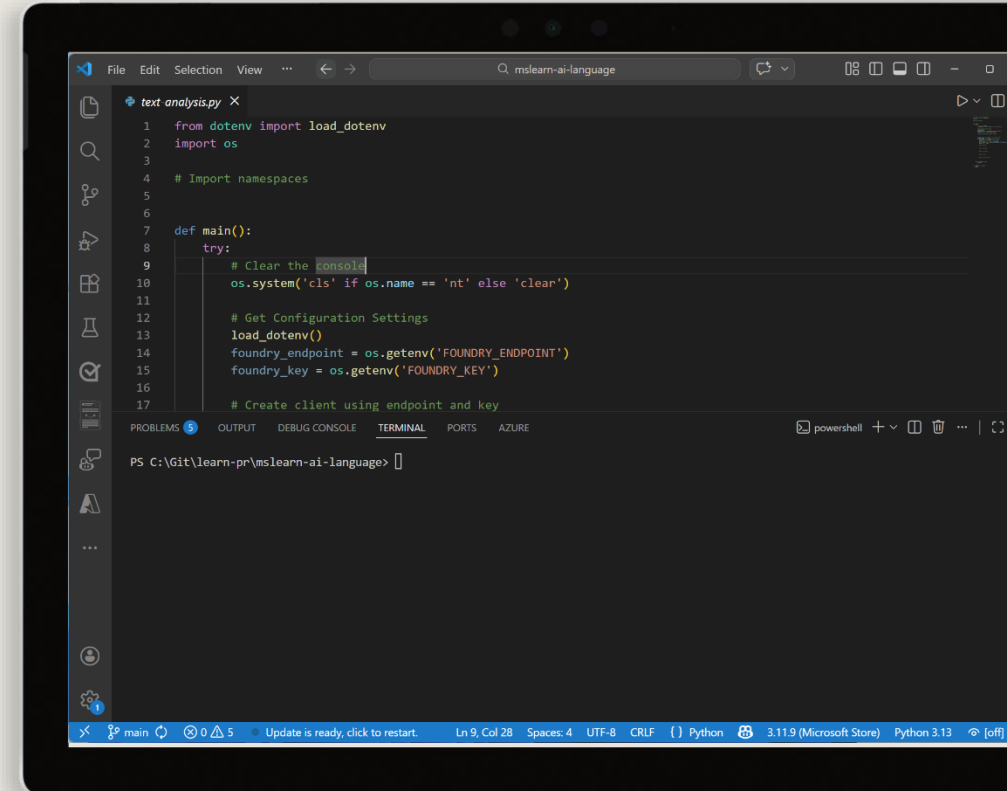
In this exercise, you'll:

- Create an agent
- Configure your data and Foundry IQ
- Test the agent in the playground
- Connect to your agent from a client application
- Test the integration

Start the exercise at:

9. Integrate an AI agent with Foundry IQ

🕒 1 h



Knowledge check



- 1** What is the primary advantage of Retrieval Augmented Generation (RAG) over simple AI agents?
 - ☐ RAG eliminates the need for large language models
 - ☒ RAG enables agents to ground responses in current organizational information and provide source transparency
 - ☐ RAG automatically retrains the language model when documents change

- 2** Which data source option provides real-time access to SharePoint content with Microsoft 365 governance?
 - ☐ SharePoint Indexed, which pre-processes content into Azure AI Search
 - ☒ SharePoint Remote, which queries SharePoint sites and libraries in real-time
 - ☐ Azure Blob Storage, which connects to SharePoint files stored as blobs

Integrate your agent with Microsoft 365

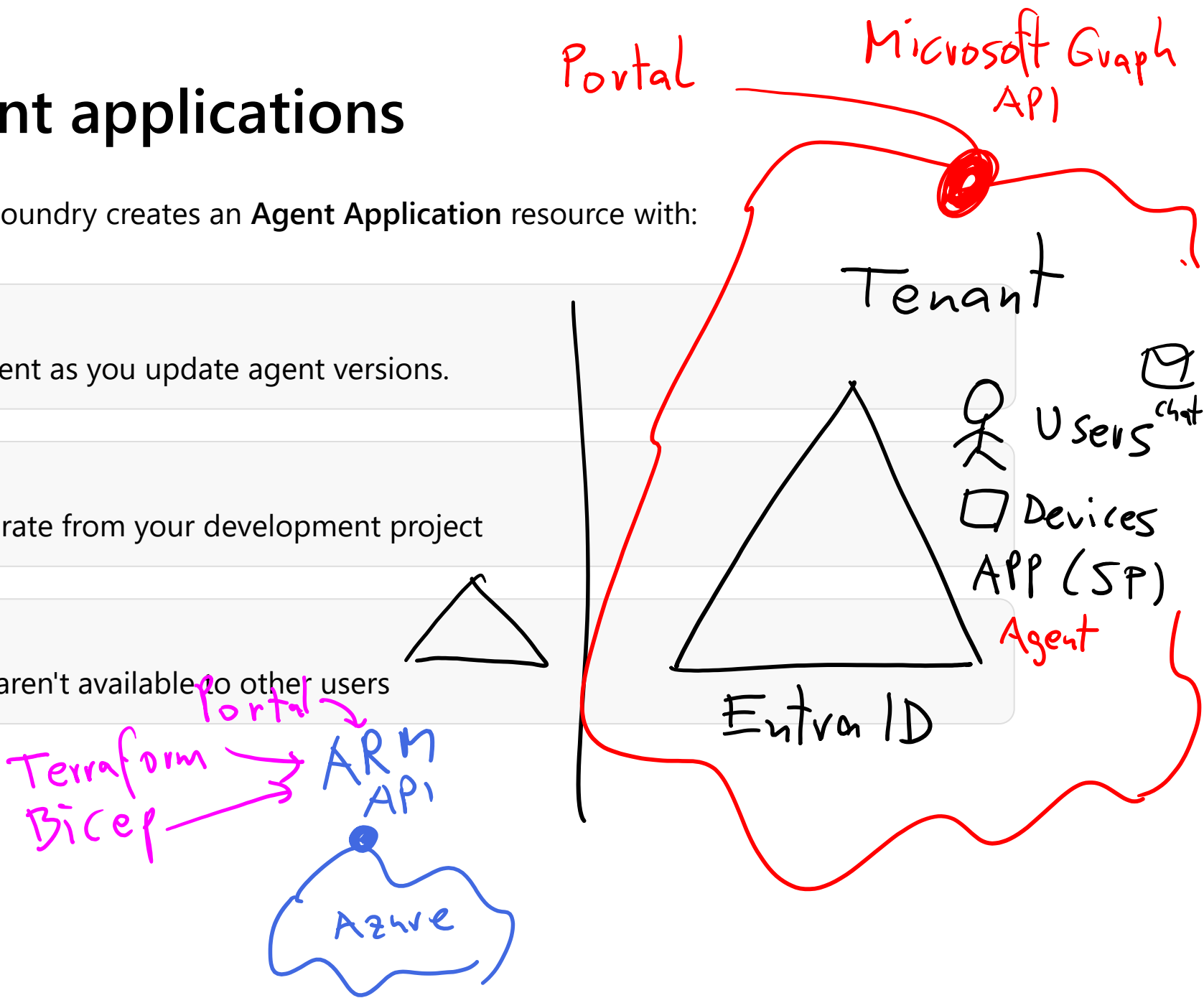
<https://aka.ms/mslearn-integrate-m365>



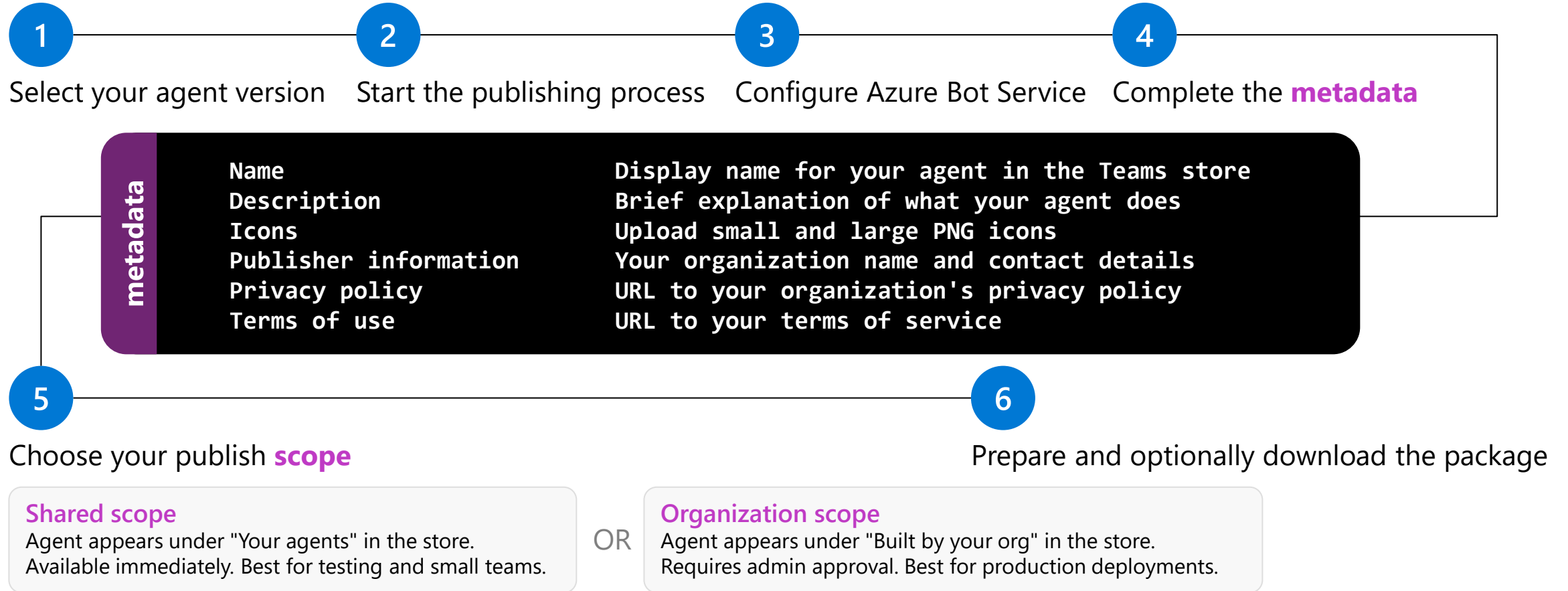
Understanding agent applications

When you publish an agent, Microsoft Foundry creates an **Agent Application** resource with:

- Dedicated invocation URL**
A stable endpoint that remains consistent as you update agent versions.
- Agent identity**
A distinct Microsoft Entra identity separate from your development project
- User data isolation**
Inputs and interactions from one user aren't available to other users



Publish your agent



Exercise

Publish a Foundry agent to Teams

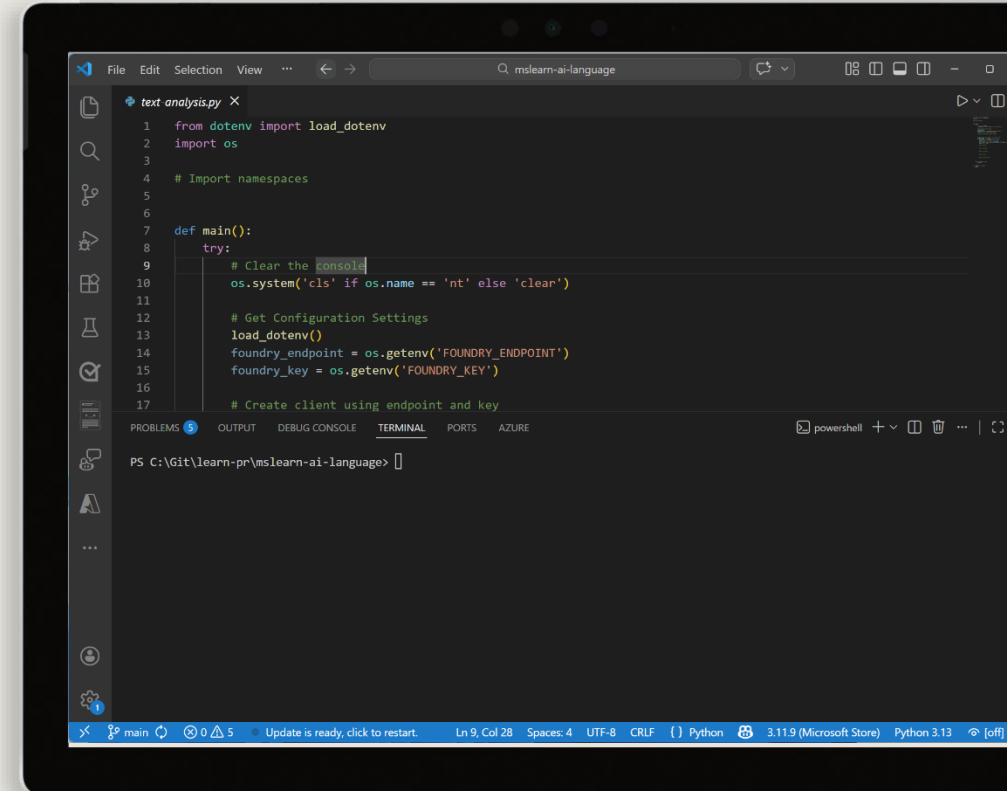
In this exercisedemo, you'll:

- Create an agent in the portal
- Add knowledge with File Search
- Publish to Microsoft Teams
- Publish to Microsoft 365 Copilot
- Update published agents

Start the exercise at:

10. Deploy agents to Microsoft Teams and Copilot

🕒 1 h



Knowledge check



1 What Azure resource does the Foundry portal automatically create when you publish an agent to Microsoft Teams?

- ☐ Azure Functions
- ☒ Azure Bot Service
- ☐ Azure Cosmos DB
- ☐ Azure Logic Apps

2 What is Microsoft Work IQ?

- ☐ A machine learning model for workplace analytics
- ☒ A CLI and MCP server that connects AI agents to Microsoft 365 data
- ☐ A replacement for Microsoft Teams
- ☐ A Visual Studio Code extension for building agents

Build agent-driven workflows using Microsoft Foundry

<https://aka.ms/mslearn-agent-workflows>



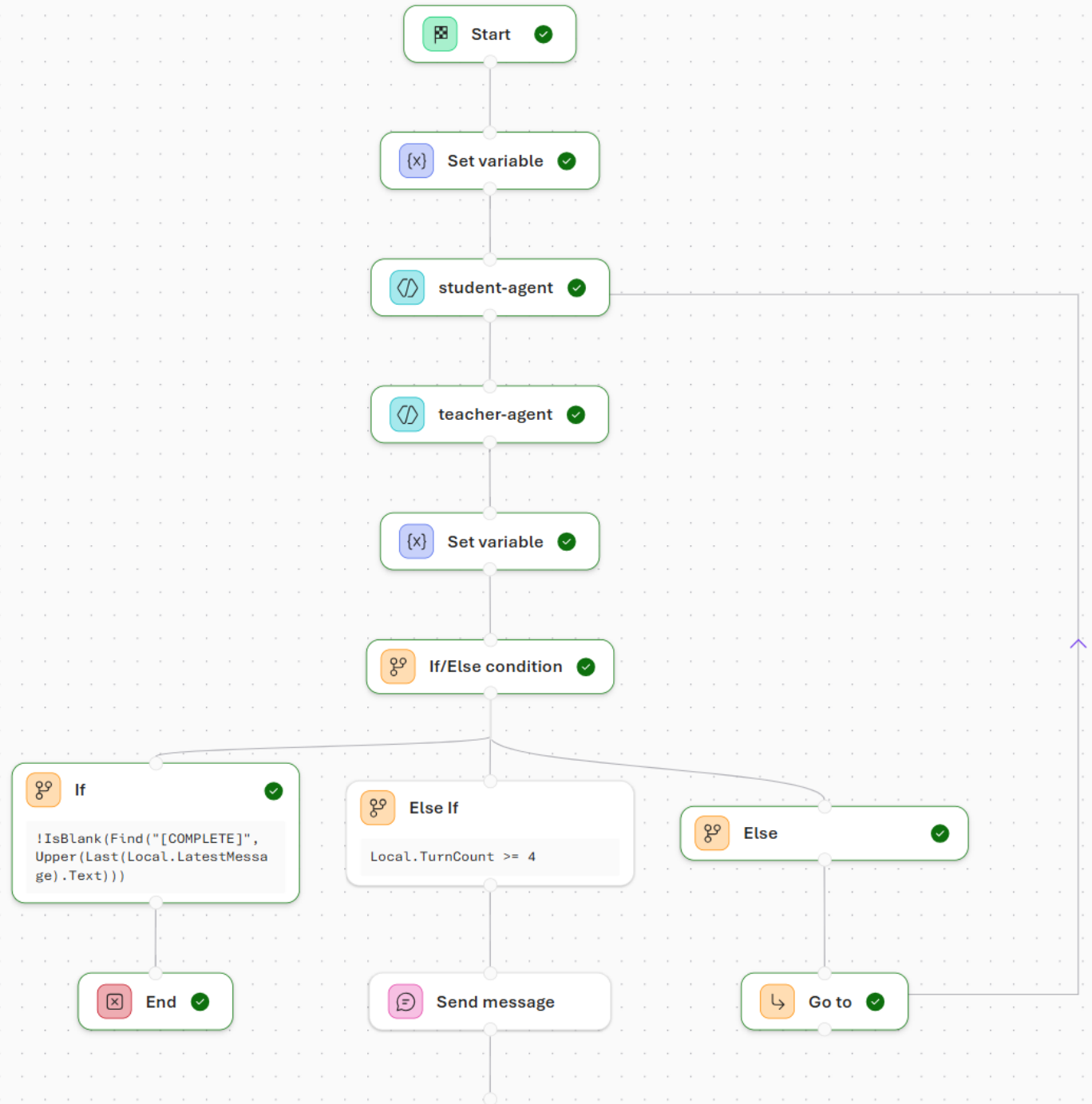
Understand workflows

Orchestrate AI-driven actions

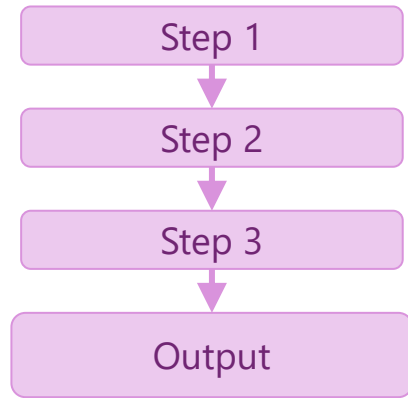
Rather than writing code, you define a sequence of steps using a visual, declarative approach that describe what should happen and when, allowing the platform to manage execution and state.

Connect the nodes

Each node performs a specific function. Some nodes invoke agents, while others evaluate conditions, manage data, or communicate with users. Together, these nodes form an execution path that determines how requests move through the system.

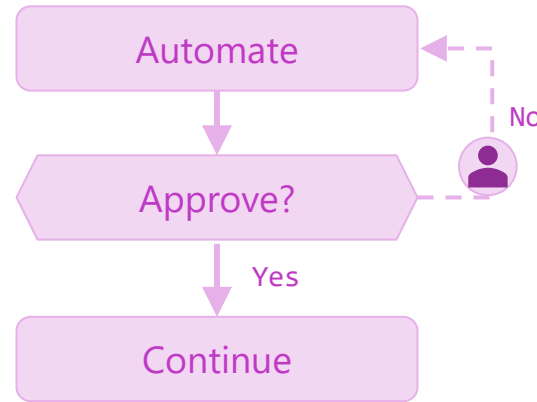


Identify workflow patterns



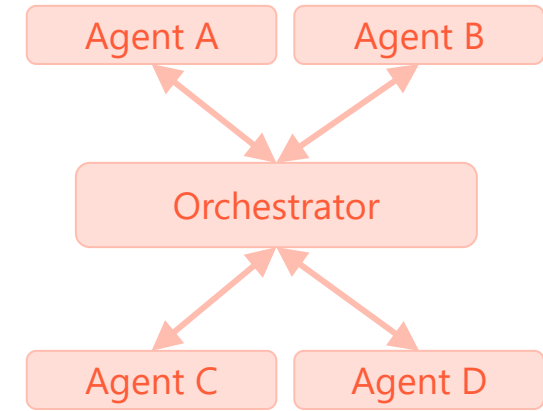
Sequential

Steps execute in a fixed order, each passing its output to the next. Best for predictable multi-stage pipelines.



Human-in-the-loop

Workflow pauses for human approval or input before continuing. Balances automation with oversight.



Group chat

Multiple specialized agents collaborate dynamically. Control shifts based on context, allowing agents to build on each other's outputs.

Explore core components of a workflow

Executors

The main workers in a workflow. Each executor receives input, performs actions, and passes outputs along — driving the workflow toward its goal.

Executors can represent AI agents or custom logic components.

Edges

Direct

One executor feeds directly into the next in sequence.

Conditional

Triggers only when a condition is met; otherwise skipped.

Switch-case

Routes to different executors based on predefined conditions

Fan-out

Sends a single message to multiple executors simultaneously.

Fan-in

Combines messages from multiple executors into one final step

Events

- **WorkflowStartedEvent**
Triggered when workflow execution begins.
- **WorkflowOutputEvent**
Emitted when the workflow produces an output.
- **WorkflowErrorEvent**
Occurs when an error is encountered during execution.
- **ExecutorInvokeEvent**
Fired when an executor starts processing a task.
- **ExecutorCompleteEvent**
Fired when an executor finishes its work.
- **RequestInfoEvent**
Logged when an external request is issued.

Exercise

Build a workflow in Microsoft Foundry

In this exercise, you'll:

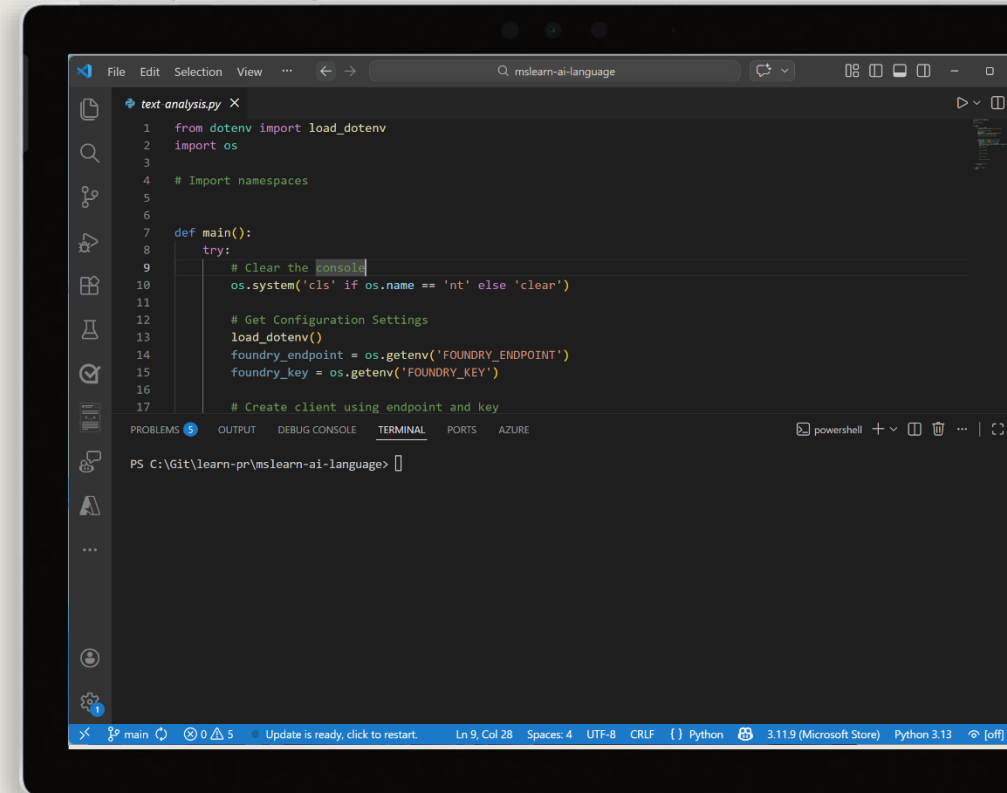
- Create a customer support triage workflow
- Preview the workflow
- Use your workflow in code

Start the exercise at:

11. Build a workflow in Microsoft Foundry



1 h



Knowledge check



1 Which type of node in a Foundry workflow is used to invoke an AI agent?

- ☐ Logic node
- ☒ Agent node
- ☐ Data transformation node

2 Which node type would you use to handle multiple items in a workflow without duplicating nodes?

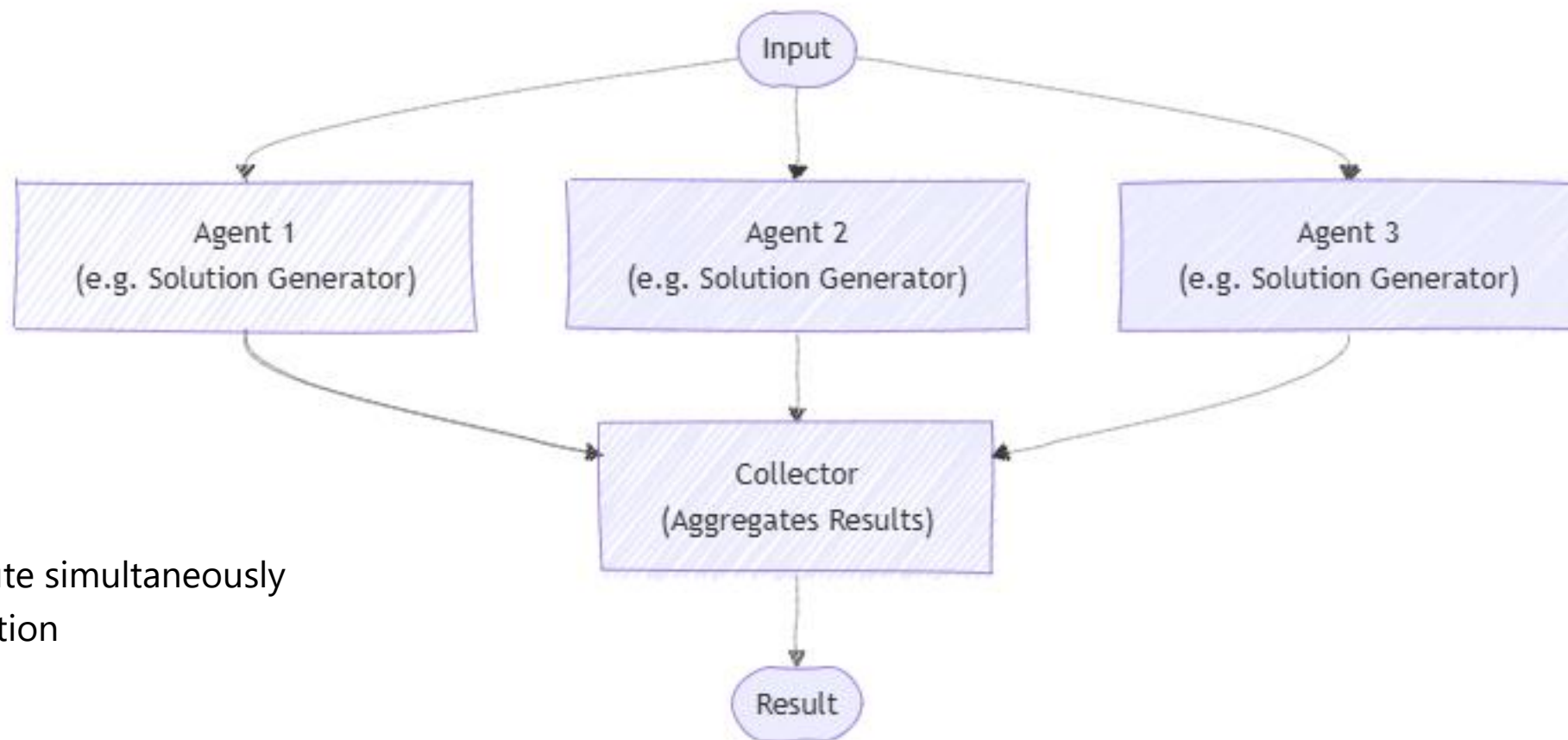
- ☐ If/Else node
- ☒ For-Each node
- ☐ Send message node

Orchestrate a multi-agent solution using the Microsoft Agent Framework

<https://aka.ms/mslearn-agent-framework-multi-agent>



Use concurrent orchestration



Run tasks in parallel

Independent agents execute simultaneously

Results merged at completion

Optimized for speed

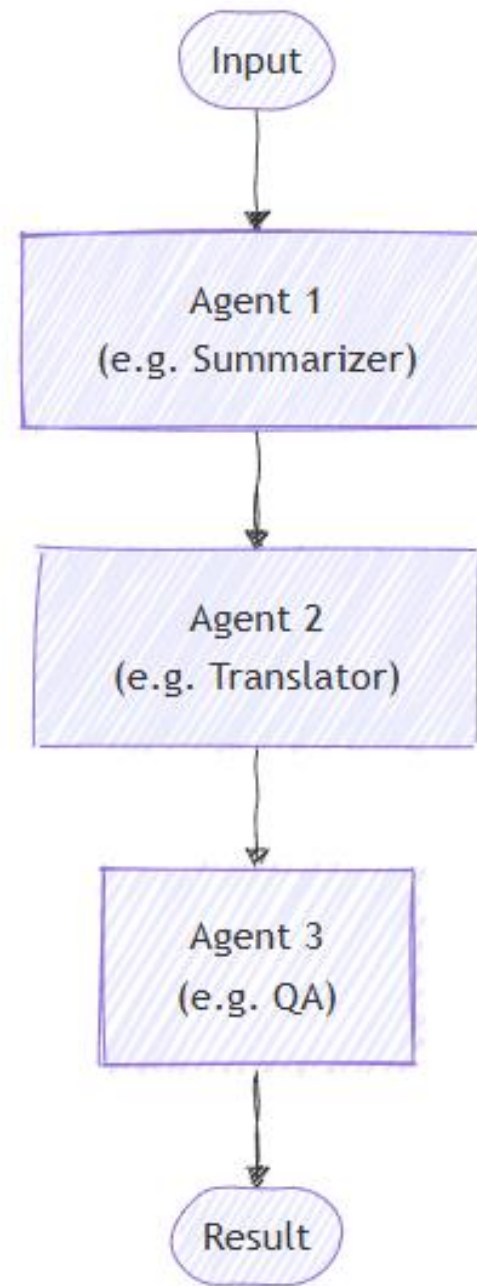
Use sequential orchestration

Step-by-step execution

Each step depends on the previous output

Predictable, ordered flow

Easy to reason about



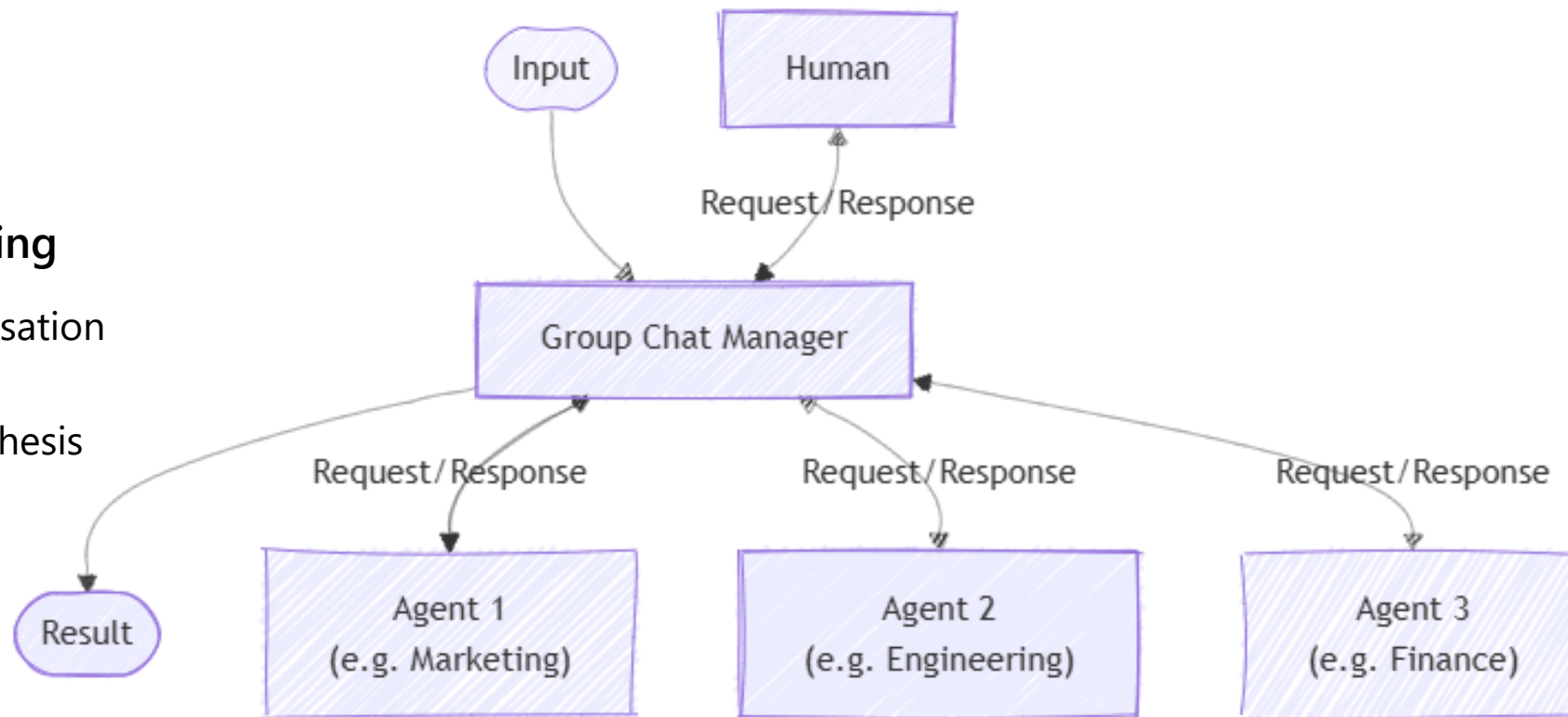
Use group chat orchestration

Collaborative agent reasoning

Agents share a common conversation

Control shifts dynamically

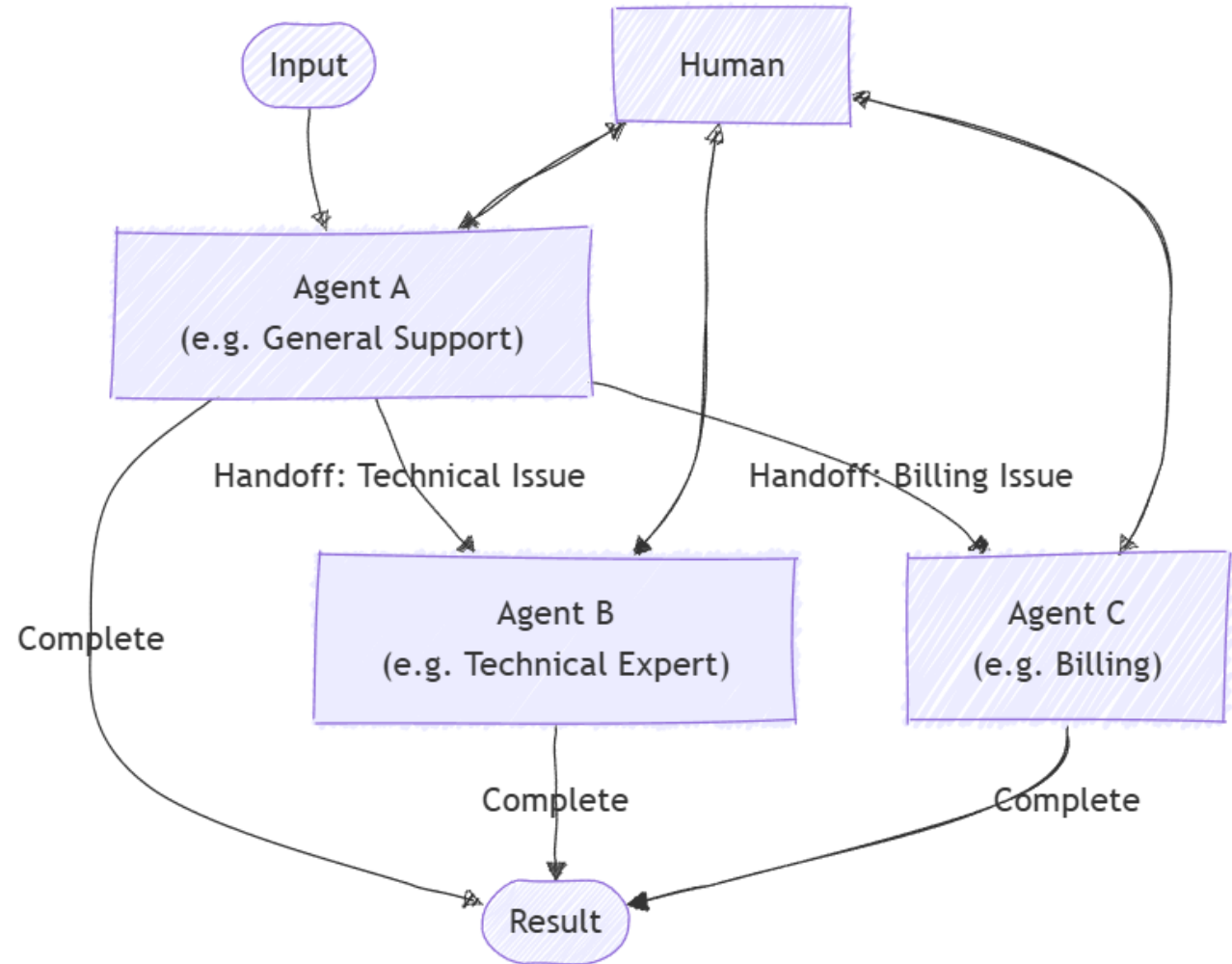
Best for brainstorming and synthesis



Use handoff orchestration

Delegate by expertise

One agent transfers control to another
Clear ownership per stage
Ideal for role-based workflows



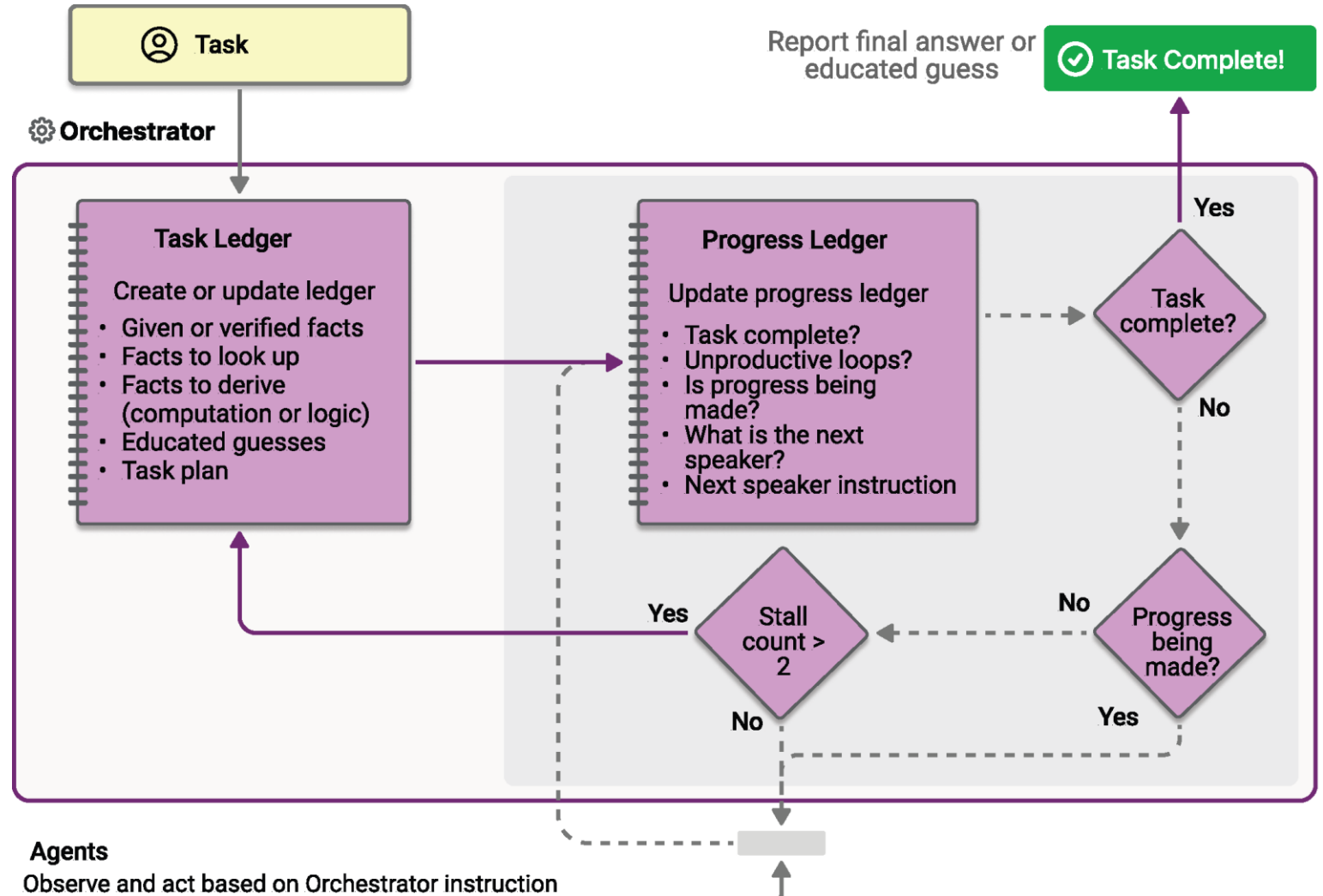
Use Magentic orchestration

Adaptive, emergent collaboration

No fixed execution order

Agents self-organize

Suited for complex, open-ended problems



Exercise

Develop a multi-agent solution

In this exercise, you'll:

- Install the Microsoft AI Toolkit VS Code extension
- Deploy a model
- Create AI agents
- Create a sequential orchestration
- Run the app

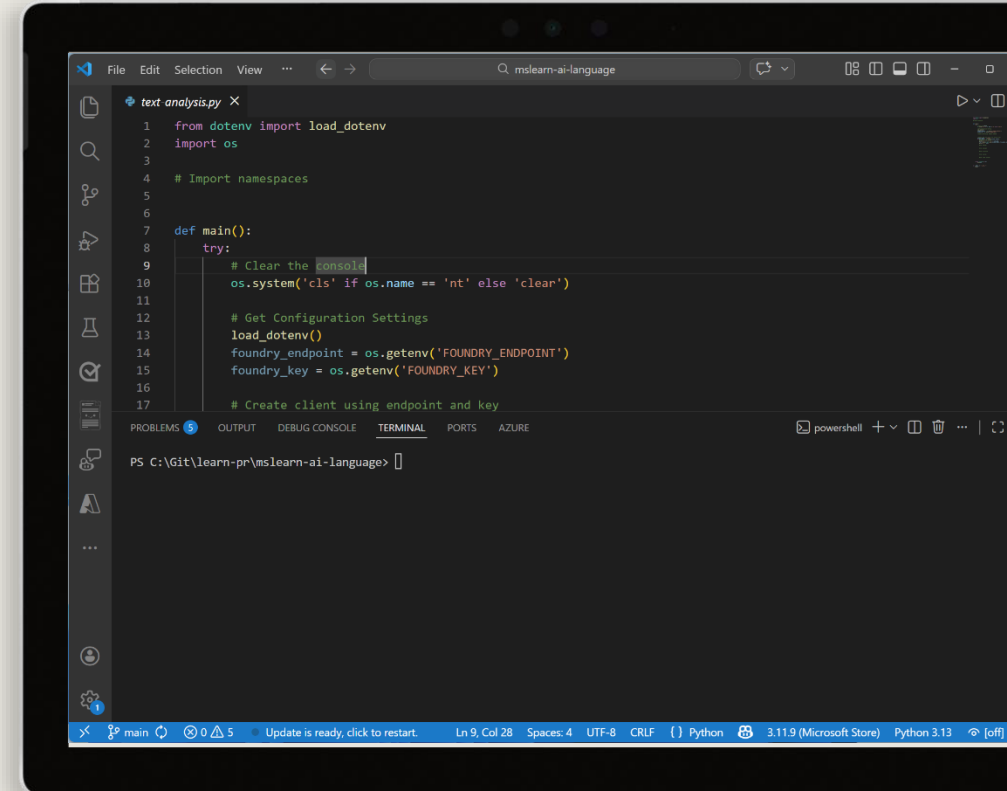
Start the exercise at:

12. Develop a multi-agent solution with Microsoft Agent Framework



1 h

0%



Knowledge check



1 What's the first step in the Microsoft Agent Framework's unified orchestration workflow?

- ☐ Select and create an orchestration pattern
- ☒ Define your agents and describe their capabilities
- ☐ Start a runtime to manage execution

2 For brainstorming and collaborative problem solving among multiple agents, which orchestration pattern is most suitable?

- ☒ Group Chat
- ☐ Magentic
- ☐ Sequential



